

UNIVERSIDAD DE CÁDIZ

FACULTAD DE CIENCIAS

GRADO EN MATEMÁTICAS

ÁRBOLES DE DECISIÓN Y RANDOM FOREST: UNA APLICACIÓN REAL

Trabajo de fin de grado presentado por

Noelia Gómez Dorado

Tutor: Alfonso Suarez Llorens

Firma del alumno Firma del tutor

Puerto Real, Cádiz, Julio de 2026

Abstract

The main objective of this work is the theoretical study and practical implementation of supervised learning algorithms, focusing the analysis on decision trees and their extensions into ensemble methods.

Following an introduction to the nature of databases in Big Data environments, the architecture of classification and regression trees is explored in depth. A technical comparison is conducted between the CHAID algorithm, based on hypothesis testing, and the CART algorithm, oriented towards impurity minimization through recursive partitioning of the predictor space.

Subsequently, the Random Forest algorithm is analyzed as an evolution of the bagging method, exploring how the use of bootstrap samples and the random selection of predictors allow for a reduction in the model's overall variance without sacrificing bias.

Finally, using the R programming environment, a real-world application is developed on a clinical dataset of breast cancer patients. Given the censored nature of the time-to-event variable, the classical decision tree framework is extended to survival analysis, building survival trees and Random Survival Forests with the log-rank splitting criterion. The results confirm the superiority of random forests in terms of generalization capacity and provide a robust inference tool for the individualized prediction of clinical relapse risk.

An appendix is included containing the proofs omitted in their respective chapters. In particular, the variance and bias properties of the forest estimator, as well as the equivalence between the Bonferroni adjustment and the Stirling numbers of the second kind, along with the additional code used to generate the figures presented in this work.

A mis padres y mi pareja

Resumen

El presente trabajo tiene como objetivo el estudio teórico y la implementación práctica de algoritmos de aprendizaje supervisado, centrando el análisis en los árboles de decisión y sus extensiones hacia métodos de ensamblado, los bosques aleatorios.

Tras una introducción a la naturaleza de las bases de datos en entornos de *Big Data*, se profundiza en la arquitectura de los árboles de clasificación y regresión. Se realiza una comparativa técnica entre el algoritmo CHAID, fundamentado en contrastes de hipótesis, y el algoritmo CART, orientado a la minimización de la impureza mediante particiones recursivas del espacio de predictores.

Posteriormente, se analiza el algoritmo de *Random Forest* como una evolución del método *bagging*, explorando cómo el uso de muestras *bootstrap* y la selección aleatoria de predictores permiten reducir la varianza global del modelo sin sacrificar el sesgo.

Finalmente, mediante el entorno de programación R, se desarrolla una aplicación real sobre un conjunto de datos clínicos de pacientes con cáncer de mama. Dada la naturaleza censurada de la variable temporal del estudio, el marco de los árboles de decisión clásicos se extiende al análisis de supervivencia, construyendo árboles de supervivencia y *Random Survival Forests* con criterio de partición *log-rank*. Los resultados confirman la superioridad de los bosques aleatorios en términos de capacidad de generalización y proporcionan una herramienta robusta para la predicción individualizada del riesgo de recaída clínica.

Se incluye un anexo donde se formalizan las demostraciones omitidas en sus capítulos correspondientes. En particular, las propiedades de varianza y sesgo

del estimador del bosque, así como la equivalencia entre el ajuste de Bonferroni y los números de Stirling de segunda especie, junto con el código adicional empleado para generar las figuras presentadas en el trabajo.

Agradecimientos

En primer lugar agradecer a Alfonso Suárez por su excelente guía y rigor académico. Le agradezco sinceramente el haber compartido su conocimiento conmigo, su disposición para orientarme en cada etapa de esta investigación y por exigirme siempre un paso más allá, ayudándome a crecer no solo como estudiante, sino también como profesional.

También me gustaría agradecer a mis padres por ser mi apoyo en los momentos difíciles y por impulsarme para alcanzar mis metas y por brindarme los valores necesarios para llegar hasta aquí.

También agradecer a mi pareja por su paciencia infinita y por ser el refugio donde descansar en los momentos de mayor presión. Gracias por celebrar mis pequeños avances, por escuchar mis dudas y por caminar a mi lado con tanto cariño durante todo este proceso.

Por último agradecer a mis compañeros, por haber hecho de este camino una experiencia compartida. Gracias por las horas de estudio, los debates, los cafés terapéuticos y por formar esa red de apoyo que convirtió los momentos de estrés en recuerdos inolvidables. Sin vuestra amistad y colaboración, este trayecto no habría sido el mismo.

Noelia Gómez Dorado

31 de Julio de 2026

Índice

1 Introducción **1**

1.1 Objetivos y estructura del trabajo 2

2 Preliminares **5**

2.1 Base de datos 5

2.2 Big Data 6

2.3 Árboles de decisión 7

2.4 Bosques aleatorios (*Random forest*) 10

3 Árboles de decisión **13**

3.1 Algoritmo CHAID 14

3.1.1 Variable Objetivo Nominal 14

3.1.2 Variable Objetivo Numérica 16

3.2 Algoritmo CART 24

3.2.1 Árboles de Regresión 26

3.2.2 Árboles de Clasificación 28

3.2.3 Criterios de parada y control del sobreajuste en CART 37

3.3 Poda 38

4 Bosques aleatorios **41**

5 Una Aplicación real **47**

5.1 Árboles de Supervivencia 49

5.1.1 Construcción del árbol de supervivencia 49

5.1.2 Predicción en los nodos terminales 50

5.1.3 Random Survival Forest 51

5.1.4 Métricas de evaluación 51
5.2 Implementación en R 51
5.2.1 Procesamiento de los datos 52
5.2.2 Ajuste del modelo: Árboles de Supervivencia 53
5.2.3 Ajuste del modelo Random Survival Forest 58
5.2.4 Creación modelo completo 61

6 Conclusiones **63**

7 Anexos **67**
7.1 Derivaciones Matemáticas y Demostraciones 67
7.1.1 Varianza del Bosque Aleatorio 67
7.1.2 Sesgo del Bosque Aleatorio 68
7.1.3 Ajuste de Bonferroni para Variables Nominales (Stirling) 68
7.2 Códigos y Scripts en R 69

Bibliografía **85**

La naturaleza está escrita en el lenguaje de las matemáticas. Los patrones de la vida no son la excepción.

Galileo Galilei

CAPITULO

1

Introducción

La vida cotidiana no es una ciencia exacta, sino un juego de probabilidades. Desde que nos despertamos, el cerebro gestiona la incertidumbre: ¿lloverá hoy? ¿Habrá tráfico?

La toma de decisiones en entornos de incertidumbre constituye uno de los pilares del progreso humano. Históricamente, esta búsqueda de orden en el caos se formalizó en el siglo XVII con el intercambio epistolar entre Blaise Pascal y Pierre de Fermat (1654), quienes sentaron las bases de la Teoría de la Probabilidad. Desde entonces, la Estadística ha evolucionado de ser una mera herramienta descriptiva a convertirse en el lenguaje formal para cuantificar lo desconocido.

La potencia de esta disciplina alcanza su máximo exponente a través de la inferencia estadística, consolidada a principios del siglo XX por figuras como Ronald A. Fisher, quien introdujo conceptos fundamentales como el diseño experimental y la máxima verosimilitud. Este proceso permite superar el análisis descriptivo para realizar generalizaciones sobre poblaciones extensas a partir de muestras limitadas. La inferencia es el motor que permite aprender de la experiencia para predecir eventos futuros, aplicada hoy en la evaluación de riesgos financieros o el diagnóstico clínico.

Los árboles de decisión representan una de las herramientas más sofisticadas de esta evolución. Aunque el concepto de ”árbol” es antiguo, su formalización matemática en la computación surgió en 1963 con el trabajo de Morgan y Sonquist (el algoritmo AID). Sin embargo, el gran hito llegaría en 1984 con la publicación de ”*Classification and Regression Trees*” (CART) por Leo Breiman, Jerome Friedman, Richard Olshen y Charles

Stone, quienes definieron la segmentación recursiva basada en medidas como la entropía o el índice de Gini. Posteriormente, en 2001, el propio Breiman revolucionaría el campo al introducir los *Random Forests*, dotando a los modelos de una robustez y capacidad de generalización sin precedentes.

Estos modelos se han vuelto indispensables en la medicina, para diagnosticar enfermedades basadas en síntomas; en las finanzas, para la concesión de créditos; o en el sector inmobiliario, permitiendo estimar el precio de una vivienda en función de su ubicación o antigüedad.

1.1 Objetivos y estructura del trabajo

El objetivo principal de esta investigación es desarrollar y validar un modelo predictivo robusto para identificar la probabilidad de supervivencia en pacientes oncológicos, con un enfoque crítico en la reducción del Error de Tipo II para minimizar los falsos negativos en el diagnóstico clínico.

Para alcanzar este fin, se plantean los siguientes objetivos específicos.

- **Comparativa algorítmica:** analizar las características metodológicas de los modelos de clasificación CHAID y CART, identificando sus fundamentos teóricos, criterios de partición y supuestos subyacentes, con el fin de justificar la elección del algoritmo más adecuado.
- **Optimización mediante conjuntos:** demostrar la superioridad de los métodos de *Ensemble Learning*, específicamente el modelo de bosques aleatorios (*Random Forest*), frente al uso de árboles de decisión individuales.
- **Incorporación del análisis de supervivencia:** extender el enfoque de árbol de decisión al contexto de datos censurados mediante la construcción de un árbol de supervivencia (*survival tree*), cuantificando la función de supervivencia estimada por Kaplan-Meier en cada nodo terminal para caracterizar el pronóstico diferencial de los subgrupos de pacientes identificados.

La estructura de este trabajo se organiza de la siguiente manera: Tras la presente introducción, el Capítulo 2 expone los conceptos matemáticos y estadísticos necesarios para el análisis. El Capítulo 3 detalla el marco teórico y el funcionamiento de los algoritmos de los árboles de decisión individuales. En el Capítulo 4 se profundiza en la teoría de los

modelos de bosques aleatorios. El Capítulo 5 recoge la metodología, el preprocesamiento de los datos y la discusión de los resultados obtenidos. El Capítulo 6 presenta las conclusiones y deducciones generales del estudio. Finalmente en el Anexo 7 se incluyen las demostraciones omitidas en los capítulos anteriores junto con el código fuente empleado para la generación de las gráficas.

CAPITULO

2

Preliminares

En esta sección se definirán los conceptos fundamentales de los árboles de decisión y los bosques aleatorios. Asimismo, se describirá la procedencia y naturaleza del conjunto de datos utilizado, detallando su estructura, las tipologías de las variables y el proceso de obtención, junto con conceptos de *Big Data*.

2.1 Base de datos

Para realizar estudios siempre es necesario disponer de una base de datos con la que trabajar. Esta base de datos se ha obtenido a partir de realizar mediciones (estudios médicos/físicos) o preguntas a un número determinado de individuos. Luego se recogen los datos de n individuos.

La base de datos se caracteriza por tener un número de filas n, donde cada fila se corresponde con un individuo al que se le ha realizado el estudio, mientras que las columnas se corresponderán con las variables, numéricas o categóricas, que han sido medidas y que dependerán del tipo de estudio y de la persona a cargo de ese mismo estudio. Por ejemplo, supóngase que un médico quiere predecir una enfermedad y decide que, con 5 variables que él ha considerado, son suficientes para poder realizar el estudio. Mientras que otro médico decide predecir la misma enfermedad, pero, en lugar de medir 5 variables, ve necesario medir 10, y que además no todas coinciden con las del médico anterior. Es por eso que el estudio dependerá de la base de datos que se tome.

En el presente trabajo se seleccionarán bases de datos que sirvan para predecir una variable de interés en función de otras, a las que llamaremos variables predictoras o covariables.

2.2 Big Data

El *Big Data* se define como el conjunto de tecnologías y metodologías diseñadas para gestionar datos que, debido a su volumen, velocidad o complejidad, superan la capacidad de las herramientas de software convencionales. Esta complejidad inherente al fenómeno del *Big Data* se articula tradicionalmente a través del modelo de las $\ll$5 V$\gg$, dimensiones que permiten desglosar los desafíos y oportunidades que presentan los datos masivos:

- **Volumen**: hace referencia a la magnitud de los datos generados, que hoy en día se miden en escalas de terabytes o petabytes, requiriendo arquitecturas de almacenamiento distribuidas.
- **Velocidad**: representa el ritmo frenético al que los datos son creados, transmitidos y procesados, exigiendo respuestas en tiempo real o casi real para que la información no pierda su vigencia.
- **Variedad**: se refiere a la heterogeneidad de las fuentes, combinando datos estructurados (como bases de datos relacionales), semiestructurados (XML, JSON) y no estructurados (imágenes, audios o texto libre).
- **Veracidad**: alude a la calidad, fiabilidad e incertidumbre intrínseca de los datos. Datos con alta veracidad son aquellos cuyo ruido o sesgo de medición es bajo.
- **Valor**: es la dimensión más estratégica, consistente en la capacidad de transformar esta materia prima informática en conocimiento útil y ventajas competitivas mediante el análisis predictivo.

Este fenómeno está estrechamente ligado al desarrollo de la Ciencia de Datos (*Data Science*) y del aprendizaje automático (*Machine Learning*), disciplinas que proporcionan las herramientas estadísticas y computacionales necesarias para extraer conocimiento útil de estos grandes volúmenes de información. En este marco, los algoritmos de aprendizaje supervisado, como los árboles de decisión y los bosques aleatorios, constituyen una de las

técnicas más empleadas para la construcción de modelos predictivos capaces de escalar a conjuntos de datos de gran tamaño y complejidad.

Un ámbito en el que el *Big Data* ha tenido especial impacto es el sector sanitario. El uso masivo de historiales clínicos electrónicos, pruebas diagnósticas y registros de seguimiento de pacientes ha dado lugar a lo que se conoce como *Big Data* biomédico, cuyo análisis permite mejorar el diagnóstico precoz, personalizar tratamientos y predecir el riesgo de recaída o progresión de enfermedades.

En este contexto, los árboles de decisión y los bosques aleatorios se posicionan como herramientas fundamentales dentro del aprendizaje supervisado, ya que son capaces de procesar gran variedad de variables y manejar grandes volúmenes de datos para extraer, en última instancia, el valor predictivo necesario para la toma de decisiones.

2.3 Árboles de decisión

Un árbol de decisión es un modelo predictivo que representa un conjunto de decisiones en forma de diagrama ramificado, como se muestra en la Figura 2.1. [1].

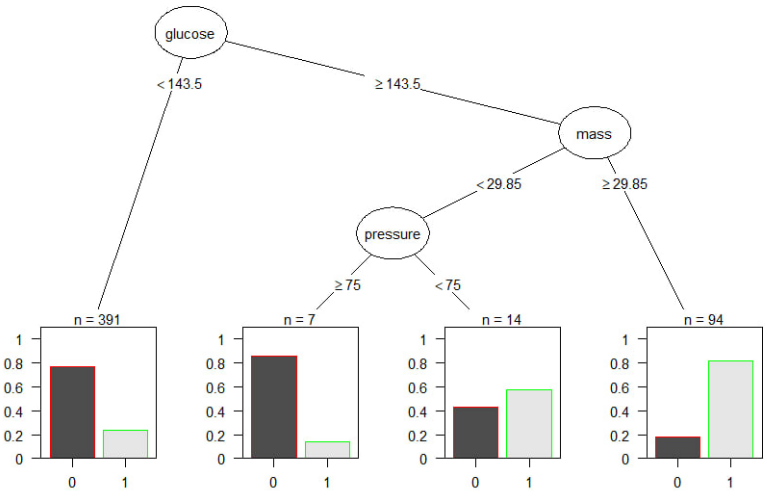


Figure 2.1: Árbol de Clasificación.

En un árbol podemos identificar tres tipos de nodos: nodo raíz (*Root Node*), nodo de decisión o ramas, y nodos hoja (*Leaf Nodes*).

[1] El código del árbol 2.1 se dejará en el Anexo 7

El nodo raíz es el nodo inicial del árbol y contiene la totalidad del conjunto de entrenamiento antes de cualquier partición. Los nodos del final, llamados nodos hoja, son los que se utilizan al final para clasificar; el resto de nodos intermedios son nodos de decisión, que se usan para saber visualmente qué recorrido está realizando el algoritmo para llegar al nodo hoja.

Para la realización de los árboles se requiere una base de datos que, como se mencionó anteriormente, se caracteriza por tener los datos de varios individuos sobre un conjunto de variables a medir. En este caso se tiene una variable que será la más importante, denominada Y, que será la variable objeto de predicción, también llamada variable respuesta. El resto serán variables predictoras que se emplean para predecir esa variable Y. Además, se tomará un porcentaje de los datos de esa base de datos: uno para entrenar el modelo y el otro para realizar el testeo, y así poder calcular posibles errores.

Para el árbol representado en la Figura 2.1, se ha usado una base de datos con pacientes que tienen diabetes frente a pacientes que no la presentan. El objetivo es predecir si un nuevo paciente puede llegar a tener diabetes, es por eso que se establece que la variable Y es la siguiente:

$$Y = \begin{cases} 1 & \text{Presenta diabetes,} \\ 0 & \text{No presenta diabetes.} \end{cases} \tag{2.1}$$

Este mismo árbol tiene como variables predictoras: número de veces embarazada (*pregnant*), función de pedigrí de diabetes (*pedigree*), edad en años (*age*), grosor del pliegue cutáneo del tríceps (*triceps*), insulina sérica a las 2 horas (*insulin*) e índice de masa corporal (*mass*). También tenemos marcadores como concentración de glucosa en plasma (*glucose*), presión arterial diastólica (*pressure*) y la variable objetivo (*diabetes*).

Luego, si se miden dichas variables a un paciente cualquiera y se introducen en el modelo, nos dará la predicción para ese paciente. Esta predicción puede ser buena o mala; se verá que con el uso de varios árboles (bosque aleatorio) se puede llegar a reducir ese error.

Una de las ventajas principales de los árboles de decisión como método de *Big Data* es que la interpretación es mucho más accesible que la de otras técnicas, como las redes neuronales, ya que proporcionan una estructura visual del modelo. Luego, realmente no

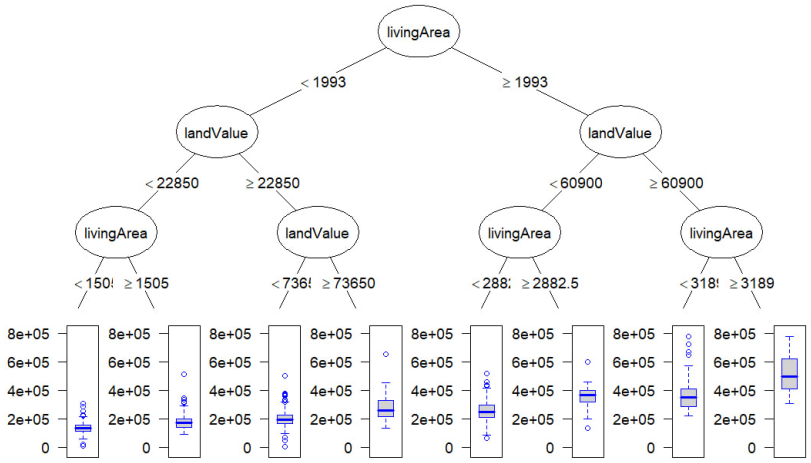


Figure 2.2: Árbol de Regresión

resulta estrictamente necesario introducir los datos del nuevo paciente en el modelo, ya que visualmente podemos llegar a la conclusión.

Dentro del modelo de los árboles de decisión existen dos tipos de árboles: los de clasificación y los de regresión. Los de clasificación se caracterizan por tener una variable para predecir que toma un conjunto de valores finitos, como por ejemplo el árbol anterior representado en la Figura 2.1, donde la variable respuesta solo toma los valores Diabetes y No Diabetes, que podemos traducir como ≪0≫ o ≪1≫. Otro ejemplo de árbol de clasificación es la clasificación de un jugador de baloncesto por categorías ≪A≫, ≪B≫, ≪C≫ y ≪D≫ en función de sus cualidades.

En el árbol de clasificación anterior, Figura 2.1, podemos ver que en los nodos terminales se tiene un valor n, que es el número de personas que hay en ese nodo, y un histograma que muestra el conjunto de personas de cada categoría que hay en ese nodo. La categoría que predice ese nodo es la que tenga un valor más grande.

El otro tipo de árbol que existe es el árbol de regresión, ver Figura 2.2 [1]. Estos árboles, por el contrario, se caracterizan porque la variable respuesta es continua.

El árbol dado en la Figura 2.2 nos muestra cómo predecir el precio de una casa en función de la edad, el número de habitaciones y de baños, el valor del suelo y los metros habitables de la vivienda (en pies cuadrados). Este árbol es de regresión porque el precio

[1] El código del árbol 2.2 se dejará en el Anexo 7

de la casa no puede tomar un conjunto de valores finitos: el precio puede tomar un número cualquiera de $\mathbb{R}$. En los nodos terminales de este árbol podemos ver la media estimada y hasta dónde puede llegar el precio.

Si se toma una casa con una superficie habitable de 1800 pies cuadrados, con tres dormitorios, 2 baños y un precio del suelo de 80000, podremos predecir el precio de la casa solo mirando el árbol, lo que nos indica un precio medio de 270 435 $.

En el Capítulo 3 se estudiarán los dos métodos más conocidos para la creación de árboles de decisión, mientras que en el Capítulo 5 se construirán árboles de decisión usando una base de datos real.

Es habitual que, después de la creación de un árbol de decisión, se realice un proceso llamado poda, ya que a veces el algoritmo del árbol suele ajustarse demasiado bien a los datos de entrenamiento, creando un modelo tan complejo y detallado que memoriza los datos en lugar de aprender los patrones generales, lo que resultará en errores predictivos. A esto se le llama sobreajuste (*overfitting*) de datos.

La poda servirá para eliminar ramas que no aportan un valor predictivo real, con el fin de mejorar el modelo con datos nuevos. Tenemos que tener cuidado, porque si el árbol es muy corto puede que no capture toda la información importante; es por eso que usaremos la poda por complejidad de costo.

Resumiendo, los árboles de decisión son una de las herramientas más potentes y versátiles en el análisis de datos moderno, destacando principalmente por su capacidad de síntesis y su naturaleza intuitiva. Se emplean fundamentalmente en los siguientes ámbitos: análisis de negocios, estrategia, *Machine Learning* (clasificación/regresión), diagnóstico médico, finanzas y evaluación de riesgo.

2.4 Bosques aleatorios (*Random forest*)

Los árboles de decisión tienen problemas de *overfitting*; esto significa que no funcionan correctamente cuando queremos hacer predicciones, ya que al momento de ser entrenados se ajustan bastante bien a los datos de entrenamiento, pero a la hora de hacer predicciones el error es bastante alto. Los bosques aleatorios (*Random forest*) lo que hacen es disminuir ese error manteniendo el buen ajuste a los datos, es decir, combinan la simplicidad de los árboles de decisión con la mejora de la exactitud.

El proceso de creación de un bosque aleatorio se basa en los árboles de decisión, pero con pequeñas modificaciones a la hora de tomar los datos de la base de datos y a la hora de construir el árbol. El proceso de creación de bosques aleatorios es conocido como *Bagging*.

Dos conceptos importantes en los bosques aleatorios son el sesgo (*bias*) y la varianza. El sesgo mide la diferencia entre el valor real y el valor promedio esperado por el modelo, mientras que la varianza mide cuán dispersos están los datos entre ellos. Para entenderlo mejor se recurre al típico ejemplo de la diana. La idea es disparar en el centro, pero, como la precisión no siempre es la mejor, se producen desviaciones que generan varianza o sesgo entre los disparos.

En la Figura 2.3 se puede apreciar una imagen gráfica de los cuatro posibles casos[1].

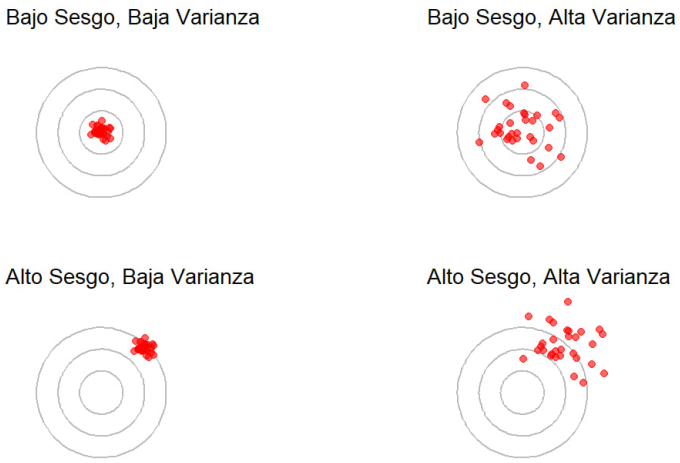


Figure 2.3: Sesgo vs Varianza.

En la diana de la primera fila, a la izquierda, se aprecia cómo los disparos se encuentran centrados y todos están muy agrupados; esto es que la varianza y el sesgo son bajos. Para la de la derecha, sin embargo, se observa que la media de los disparos está en el centro, pero los disparos individuales se dispersan ampliamente; esto implica que la varianza ha aumentado, por eso están más dispersos.

Para las dos dianas de abajo se observa que en la diana de la izquierda los disparos están agrupados, pero lejos del centro; luego, lo que ha aumentado ahora es el sesgo, ya que al estar los datos muy juntos tenemos que la varianza es pequeña, mientras que en la

[1]El código de la figura 2.3 se dejará en el Anexo 7

imagen contigua el sesgo y la varianza aumentan, proporcionando disparos lejos del centro y lejos entre ellos.

En el contexto de bosques aleatorios, el sesgo se define como la diferencia entre la esperanza de la predicción del modelo y el valor verdadero:

$$\text{Sesgo}[\hat{f}(x)] \;=\; \mathbb{E}[\hat{f}(x)] - f(x),$$

donde la esperanza se toma sobre todas las posibles muestras de entrenamiento. La varianza mide cuánto fluctúa $\hat{f}(x)$ alrededor de esa esperanza:

$$\text{Var}[\hat{f}(x)] \;=\; \mathbb{E}\Big[\,(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2\,\Big].$$

Intuitivamente, el sesgo refleja un error sistemático del modelo, mientras que la varianza refleja su sensibilidad a perturbaciones de los datos de entrenamiento.

Por último, cabe añadir que un único árbol de decisión tiende a tener un bajo sesgo pero una alta varianza, ya que se ajusta demasiado a los datos de entrenamiento. Se analizará en el Capítulo 4 que, al combinar muchos árboles en un bosque aleatorio, se logra promediar esos errores y reducir la varianza global sin sacrificar el sesgo. Además, en el Capítulo 5 se desarrollará un bosque aleatorio usando la misma base de datos empleada para los árboles individuales, para posteriormente analizar los resultados y poder llegar a conclusiones en el Capítulo 6.

CAPITULO

3

Árboles de decisión

Como se expuso en el Capítulo 2, los árboles de decisión facilitan la toma de decisiones basada en un conjunto de datos con el que se ha entrenado el modelo. En este capítulo se analizan los dos algoritmos usados en la creación de estos árboles.

Para la creación de estos modelos se utilizan habitualmente dos algoritmos: CART y CHAID.

Antes de detallar los modelos, se asume la existencia de p variables $X_1, X_2, ..., X_p$ y una variable dependiente Y que es la que queremos predecir. Además, se dispone de N observaciones independientes para el entrenamiento (que incluyen los valores de X_i y el valor objetivo Y), representadas de la siguiente manera: (x_i, y_i), para $i = 1, \ldots, N$, donde $x_i = (x_{i1}, x_{i2}, \ldots, x_{ip})$ es el vector de las p covariables observadas en el individuo i-ésimo.

El conjunto de datos completo está formado por las observaciones (x_i, y_i). Para construir y evaluar el modelo, este conjunto se divide en dos grupos: un 90% de los datos se destina a entrenar el modelo y crear el árbol, mientras que el 10% restante tiene como objetivo evaluar su capacidad para predecir correctamente.

La idea de los algoritmos consiste en encontrar, en cada paso, qué variable y qué valor generan la mejor partición de los datos, creando así una estructura ramificada. Para el algoritmo CHAID se emplea la prueba chi-cuadrado, mientras que para el algoritmo CART se evalúa el error (en árboles de regresión) y el índice de Gini o la entropía (en árboles de clasificación). Estos conceptos se desarrollan a lo largo del capítulo.

3.1 Algoritmo CHAID

Este algoritmo se utiliza cuando la mayoría de los datos son de naturaleza categórica y el objetivo es la interpretación o investigación estadística, ya que se fundamenta en contrastes de hipótesis y el estadístico Chi-cuadrado.

El nombre proviene de *Chi-square Automatic Interaction Detection* (detección automática de interacciones mediante chi-cuadrado), ver más detalles en [1].

3.1.1 Variable Objetivo Nominal

A continuación, se describe el procedimiento cuando la variable objetivo es categórica.

En primer lugar, se crean p tablas de contingencia, una por cada variable predictora. Una tabla de contingencia muestra las frecuencias observadas totales y las frecuencias para cada valor de la variable objetivo.

Se considera la siguiente tabla de valores, con n individuos, dos variables predictoras X_1, X_2 y la variable objetivo Y.

Además, la variable X_1 tiene 2 categorías, C_{11}, C_{12}; la variable X_2 tiene 3 categorías, C_{21}, C_{22}, C_{23}; y la variable Y tiene 2 categorías, Y_1, Y_2. Cabe aclarar que, si alguna variable predictora es numérica, se tendrá que realizar primero intervalos para así obtener las categorías.

Table 3.1: Tabla de datos.

ID	X_1	X_2	Y
1	x_{11}	x_{12}	y_1
2	x_{21}	x_{22}	y_2
$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	x_{n1}	x_{n2}	y_n

Por lo tanto, las tablas de contingencia serían:

Table 3.2: Tabla de Contingencia X_1

X_1	Y_1	Y_2	**Suma**
C_{11}	O_{111}	O_{112}	$O_{11.}$
C_{12}	O_{121}	O_{122}	$O_{12.}$
Suma	$O_{1.1}$	$O_{1.2}$	$O_{1..}$

Table 3.3: Tabla de Contingencia X_2.

X_2	Y_1	Y_2	**Suma**
C_{21}	O_{211}	O_{212}	$O_{21.}$
C_{22}	O_{221}	O_{222}	$O_{22.}$
C_{23}	O_{231}	O_{232}	$O_{23.}$
Suma	$O_{2.1}$	$O_{2.2}$	$O_{2..}$

donde O_{ijk} es la frecuencia observada de la variable i, para la categoría j, para la categoría de la variable objetivo k, $O_{ij.} = \sum_{k=1}^{2} O_{ijk}$, y $O_{i.k} = \sum_{j=1}^{J} O_{ijk}$. En este caso J se corresponde al número de categorías de la variable.

Se plantea, por tanto, el siguiente contraste de hipótesis:

$$\begin{cases} H_0 : \text{No existe una relación significativa, (independientes)} \\ H_1 : \text{Existe relación significativa.} \end{cases} \tag{3.1}$$

Donde el contraste 3.1 se resuelve con el clásico estadístico de chi-cuadrado de Pearson:

$$\chi^2 = \sum_{j,k} \frac{(O_{ijk} - e_{ijk})^2}{e_{ijk}},$$

donde O_{ijK} son las frecuencias observadas y e_{ijk} las frecuencias esperadas; estas se calculan como $e_{ijk} = \frac{O_{ij.} \times O_{i.k}}{O_{..}}$.

Luego, para saber cuál es la variable y el valor más óptimo para realizar la división, calculamos el p-valor del estadístico y se escoge el más pequeño que sea inferior al nivel de significación $\alpha = 0.05$. De forma equivalente, este contraste puede resolverse comparando el estadístico χ^2 calculado con el valor crítico $\chi^2_{\alpha,\, gl}$ de la distribución chi-cuadrado, donde gl son los grados de libertad asociados a la tabla de contingencia correspondiente, calculados como $gl = (\text{número de filas} - 1) \times (\text{número de columnas} - 1)$. Se rechaza H_0 si $\chi^2 > \chi^2_{\alpha,\, gl}$, lo cual es equivalente a obtener un p-valor inferior al nivel de significación α.

Además, antes de realizar las divisiones, es posible estudiar si resulta procedente agrupar algunas categorías. Para ello, se emplea el mismo contraste de hipótesis, aunque en este caso, con el fin de agrupar categorías, se analiza el p-valor de la tabla de contingencia correspondiente a ese par de categorías. Estas se agrupan si el p-valor del estadístico es superior al nivel de significación $\alpha = 0.05$. Este proceso queda resumido en el algoritmo 1. Una vez obtenidas la variable y la categoría óptimas para realizar la división, la base

de datos original queda particionada en dos subconjuntos, correspondientes a los nodos hijos generados. Sobre cada uno de estos subconjuntos se vuelve a aplicar el mismo procedimiento de forma recursiva, es decir, se repite el proceso de análisis de las tablas de contingencia y selección de la variable óptima, hasta que se alcance alguno de los criterios de parada del algoritmo.

Algorithm 1 Algoritmo CHAID para variables objetivo nominales

Se asume la existencia de p variables predictoras, donde cada variable i cuenta con i_n categorías

1. En primer lugar, se analiza la posibilidad de agrupar categorías. Para ello, se generan las matrices de contingencia entre cada variable predictora i y la variable objetivo, cuyas categorías se denotan como j y k, respectivamente, con el fin de aplicar el estadístico:
$$\chi^2 = \sum_{j,k} \frac{(O_{ijk} - e_{ijk})^2}{e_{ijk}},$$
donde las categorías se agrupan si el p-valor del estadístico es superior al nivel de significación $\alpha = 0.05$.

2. A continuación, se inicia el proceso de división. Para cada variable predictora i, se construyen las tablas de contingencia y se calcula el estadístico correspondiente junto con sus respectivos p-valores:
$$\chi^2 = \sum_{j,k} \frac{(O_{ijk} - e_{ijk})^2}{e_{ijk}}.$$
Se selecciona la variable i que presente el menor p-valor, siempre que este sea inferior al nivel de significación $\alpha = 0.05$.

3.1.2 Variable Objetivo Numérica

Si la variable objetivo es de tipo numérico, no es posible emplear matrices de contingencia ni procede la segmentación en intervalos, dado que tanto dichas matrices como el test chi-cuadrado están diseñados para el conteo de frecuencias en categorías. La conversión de una variable numérica en intervalos conlleva una pérdida de información estadística valiosa, tal como la varianza o la media exacta.

No obstante, la situación es distinta cuando la variable numérica es una predictora: en este caso, al igual que ocurre con una variable objetivo nominal, sí resulta necesario dividir

la variable predictora en intervalos para poder generar las categorías sobre las que aplicar el algoritmo.

En este escenario, se debe utilizar el test de Snedecor, mediante el cual se plantea un contraste de tipo ANOVA de un factor. Concretamente, dada una variable predictora numérica i dividida en k intervalos, se contrasta si las medias de la variable objetivo Y difieren entre dichos intervalos:

$$\begin{cases} H_0 : \mu_{i1} = \mu_{i2} = ... = \mu_{ik}, \\ H_1 : \text{al menos una media difiere}, \end{cases} \tag{3.2}$$

donde μ_{ij} es la media de la variable objetivo en el intervalo j de la variable i. El estadístico es:

$$F = \frac{B(n-k)}{W(k-1)}. \tag{3.3}$$

Además las expresiones de B y W son las siguientes:

$$B = \sum_{i=1}^{k} n_i(\bar{y}_i - \bar{y})^2 \quad \text{y} \quad W = \sum_{i=1}^{k}\sum_{j=1}^{n_i}(y_{ij} - \bar{y}_i)^2,$$

donde:

- B: suma de cuadrados entre grupos (varianza explicada).
- W: suma de cuadrados intra-grupos (varianza no explicada o del error).
- k: número total de grupos o categorías resultantes de la división.
- n: tamaño total de la muestra en el nodo analizado.
- n_i: número de observaciones dentro del grupo i.
- y_{ij}: valor de la variable objetivo para la observación j en el grupo i.
- $\bar{y}_i$: media de la variable objetivo calculada para el grupo i.
- $\bar{y}$: media global de la variable objetivo en el nodo padre (antes de la división).

Al igual que ocurre con las variables predictoras categóricas, el proceso de creación de intervalos para una variable numérica es iterativo: se parte de un número inicial de intervalos y, en cada paso, se evalúa mediante el test F si dos intervalos adyacentes pueden fusionarse, es decir, si sus medias de la variable objetivo no difieren de forma significativa. Este proceso de fusión se repite hasta que ya no sea posible unir más intervalos sin superar el nivel de significación $\alpha = 0.05$, obteniendo así la discretización final de la variable predictora numérica.

En este caso, se utiliza el test de Snedecor para medir la varianza, puesto que una división óptima es aquella que genera grupos donde los valores de Y presentan una alta similitud interna (baja varianza interna), pero muestran diferencias significativas entre los distintos grupos (alta varianza entre grupos).

Posteriormente, a fin de determinar la mejor variable y el punto de corte para la división, se calculan el valor del estadístico y el p-valor, seleccionando aquel p-valor que resulte ser el más pequeño. Asimismo, es posible agrupar las variables si el p-valor obtenido es superior al nivel de significación $\alpha = 0.05$.

Este procedimiento queda resumido en el algoritmo 2

Algorithm 2 Algoritmo CHAID para variables objetivo continuas

Se asume la existencia de p variables predictoras, donde cada variable i dispone de i_n categorías.

1. En primer lugar, se analiza la posibilidad de agrupar las categorías de los predictores. Para este fin, se emplea el estadístico F de Snedecor:

$$F = \frac{B(n-k)}{W(k-1)},$$

   donde las variables se agrupan si el p-valor del estadístico resulta superior al nivel de significación $\alpha = 0.05$.

2. Posteriormente, se inicia el proceso de división. Para cada variable predictora i, se calcula el estadístico F de Snedecor junto con sus respectivos p-valores:

$$F = \frac{B(n-k)}{W(k-1)}.$$

   Se selecciona la variable i que presente el menor p-valor, siempre que este sea inferior al nivel de significación $\alpha = 0.05$.

Además, debido a que el algoritmo CHAID realiza múltiples comparaciones para cada variable predictora en su proceso de segmentación, existe un riesgo elevado de cometer un error de Tipo I (falsos positivos). Para mitigar este efecto, se aplica el ajuste de Bonferroni. Este ajuste consiste en multiplicar el p-valor obtenido por el número de posibles particiones o fusiones de categorías de la variable evaluada, siguiendo la propuesta original descrita en [1]:

$$p_{\text{ajustado}} = p \times B, \tag{3.4}$$

donde B es el factor de Bonferroni. Este multiplicador depende del número de categorías originales (k), el número de categorías finales tras la agrupación (c) y la naturaleza de la variable predictora, tal como se detalla en la documentación técnica de [2]:

- **Variables Ordinales:** al estar restringidas a agrupaciones de categorías adyacentes, el número de combinaciones es reducido, resultando en un factor de corrección menor:

$$B = \binom{k-1}{c-1}. \tag{3.5}$$

- **Variables Nominales:** dado que no existe un orden predefinido, cualquier combinación de categorías es posible. En este caso, el factor B se deriva de los números de Stirling de segunda especie, que contabilizan las formas de particionar un conjunto de k elementos en c subconjuntos no vacíos:

$$B = \sum_{j=0}^{c-1} (-1)^j \frac{(c-j)^k}{j!(c-j)!}. \tag{3.6}$$

  La demostración de la equivalencia entre esta sumatoria y la función de Stirling se detalla en el Anexo 7.

Solo si este p-valor ajustado es inferior al nivel de significación α, se considera que la variable es apta para realizar la división del nodo.

Table 3.4: Resumen del Algoritmo CHAID

Característica	Objetivo Categórica	Objetivo Numérica
Herramienta de análisis	Tablas de contingencia	Comparación de medias
Estadístico de contraste	Chi-cuadrado (χ^2) de Pearson	F de Snedecor (ANOVA)
Hipótesis Nula (H_0)	Independencia estadística	Igualdad de medias ($\mu_1 = \mu_2$)
Criterio de Agrupación	p-valor $> \alpha$	p-valor $> \alpha$
Criterio de División	p-valor más pequeño $< \alpha$	p-valor más pequeño $< \alpha$

A continuación, se plantea un ejemplo en el que se detalla la obtención de la primera variable de división; es decir, se identifica la variable que actuará como nodo raíz. Además, se ilustra previamente un caso de agrupación de categorías.

Se considera el siguiente conjunto de datos de entrenamiento para predecir si un cliente adquirirá un vehículo en función de su salario y su conciencia ecológica.

Table 3.5: Datos ejemplo del algoritmo CHAID.

ID	Salario	Conciencia Eco.	¿Compra Coche?
1	Alto	Alta	Sí
2	Medio	Alta	Sí
3	Bajo	Baja	No
4	Medio	Alta	Sí
5	Alto	Baja	Sí
6	Medio	Baja	No
7	Bajo	Alta	No
8	Medio	Alta	Sí
9	Bajo	Alta	Sí
10	Alto	Alta	Sí

Table 3.6: Contingencia Salario Bajo vs. Medio

Salario	Si compra	No compra	Total
Bajo	1	2	3
Medio	3	1	4
Total	4	3	7

Como las variables son categóricas, se usa el estadístico chi-cuadrado de Pearson.

Se calculan los valores O_{ijk} y e_{ijk} para cada j, k, donde el subíndice i hace referencia a la variable analizada. En este análisis, para la variable Salario se asigna $i = 1$, mientras que para Conciencia Ecológica se denota como $i = 2$. Se define que, cuando $j = 1$, el salario es bajo, y cuando $j = 2$, es medio. Asimismo, el subíndice $k = 1$ indica que el cliente adquiere el vehículo y $k = 2$ indica que no se realiza la compra.

Table 3.7: Valores observados y esperados para la variable Salario

j	k	O_{1jk}	e_{1jk}
1	1	1	$12/7$
1	2	2	$9/7$
2	1	3	$16/7$
2	2	1	$12/7$

Al aplicar la fórmula del estadístico χ^2:

$$\chi^2 = \frac{(1-1.71)^2}{1.71} + \frac{(2-1.29)^2}{1.29} + \frac{(3-2.28)^2}{2.28} + \frac{(1-1.71)^2}{1.71} \approx 1.215.$$

Para calcular los grados de libertad de la chi-cuadrado, se aplica la fórmula $gl = (r-1) \times (c-1)$. Mediante el lenguaje de programación R, es posible obtener el p-valor con el comando `pchisq(1.215, df=1, lower.tail = FALSE)`, el cual devuelve un valor de 0.270. Es necesario recordar que este valor no es el definitivo, ya que requiere un ajuste posterior.

En esta fase, el ajuste de Bonferroni se aplica multiplicando el resultado por el número de combinaciones posibles de fusión para una variable de c categorías. En este caso particular, para $c = 2$ ($k = 3$), el ajuste de Bonferroni se define como:

$$B = \sum_{j=0}^{c-1} (-1)^j \frac{(c-j)^k}{j!(c-j)!} = \frac{(2-0)^3}{0!(2-0)!} - \frac{(2-1)^3}{1!(2-1)!} = 3.$$

Luego, el p-valor ajustado es de:

$$p_{ajustado} = 0.273 \times 3 = 0.819.$$

Luego, como el p-valor es mayor al nivel de significación 0.05 ($0.819 > 0.05$), se concluye que no existe una relación significativa entre esas dos categorías, luego se pueden agrupar en una sola.

A continuación, se determina qué variable funcionará como nodo raíz; es decir, en cuál se llevará a cabo la división. Se inicia el proceso calculando el p-valor (crudo) de la variable salario.

Table 3.8: Datos nivel salarial

Salario	Si compra	No compra	Total
Medio-Bajo	4	3	7
Alto	3	0	3
Total	7	3	10

Como las variables son categóricas, también se usa el estadístico chi-cuadrado de Pearson.

Se calculan los valores O_{ijk} y e_{ijk} para cada j, k, manteniendo la asignación de $i = 1$ empleada anteriormente. Se define que para el índice $j = 1$ el salario es medio-bajo, mientras que para $j = 2$ es alto. Asimismo, el valor $k = 1$ indica que se adquiere el vehículo, mientras que $k = 2$ representa que no se realiza la compra.

Table 3.9: Valores observados y esperados para la variable Salario

j	k	O_{1jk}	e_{1jk}
1	1	4	$49/10$
1	2	3	$21/10$
2	1	3	$21/10$
2	2	0	$9/10$

Al aplicar la fórmula del estadístico χ^2:

$$\chi^2 = \frac{(4-4.9)^2}{4.9} + \frac{(3-2.1)^2}{2.1} + \frac{(3-2.1)^2}{2.1} + \frac{(0-0.9)^2}{0.9} \approx 1.836.$$

Para calcular los grados de libertad de la chi-cuadrado, se aplica la fórmula $gl = (r-1) \times (c-1)$. Mediante el lenguaje de programación R, es posible obtener el p-valor con el comando `pchisq(1.836, df=1, lower.tail = FALSE)`, el cual devuelve un valor de 0.175.

Al haber realizado una fusión previa, el multiplicador de Bonferroni para esta rama es:

$$B = \sum_{j=0}^{c-1} (-1)^j \frac{(c-j)^k}{j!(c-j)!} = \frac{(2-0)^3}{0!(2-0)!} - \frac{(2-1)^3}{1!(2-1)!} = 3,$$

donde $k = 3$ y $c = 2$, y concluimos que:

$$p_{ajust-Salario} = 0.175 \times 3 = \mathbf{0.525}.$$

A continuación, se calcula el valor del estimador de la otra variable, Conciencia ecológica.

Table 3.10: Datos nivel Conciencia ecológica

Conc.Ecológica	Si compra	No compra	Total
Baja	1	2	3
Alta	6	1	7
Total	7	3	10

Usando el estadístico Chi-cuadrado, se calculan los valores O_{ijk} y e_{ijk} para cada j, k, asignando en este caso el valor $i = 2$. Se establece que, cuando $j = 1$, la conciencia ecológica es baja, mientras que $j = 2$ corresponde a una conciencia alta. Asimismo, el valor $k = 1$ indica que se adquiere el vehículo, mientras que $k = 2$ representa que no se realiza la compra.

Table 3.11: Valores observados y esperados para la variable Conciencia Ecológica

j	k	O_{2jk}	e_{2jk}
1	1	1	$21/10$
1	2	2	$9/10$
2	1	6	$49/10$
2	2	1	$21/10$

Al aplicar la fórmula del estadístico χ^2:

$$\chi^2 = \frac{(6-4.9)^2}{4.9} + \frac{(1-2.1)^2}{2.1} + \frac{(1-2.1)^2}{2.1} + \frac{(2-0.9)^2}{0.9} \approx 2.743.$$

Para calcular los grados de libertad de la prueba chi-cuadrado, se aplica la expresión $gl = (r-1) \times (c-1)$. Mediante el lenguaje de programación R, es posible obtener el p-valor con el comando `pchisq(2.743, df=1, lower.tail = FALSE)`, el cual devuelve un valor de 0.09768.

En este escenario, el multiplicador de Bonferroni para esta rama se define como:

$$B = \sum_{j=0}^{c-1} (-1)^j \frac{(c-j)^k}{j!(c-j)!} = \frac{(2-0)^2}{0!(2-0)!} - \frac{(2-1)^2}{1!(2-1)!} = 1,$$

donde $k = 2$ y $c = 2$; por lo tanto, se tiene que:

$$p_{ajust-Conciencia} = 0.0977 \times 1 = \mathbf{0.0977}.$$

Debido a que el p-valor de la variable conciencia ecológica es el más reducido, se selecciona dicha variable para conformar el nodo raíz.

Una vez ejecutada la primera división, los datos se distribuyen en dos grupos: aquellos con una conciencia ecológica alta y aquellos con una conciencia baja. Por lo tanto, se obtienen los siguientes subconjuntos de datos.

Table 3.12: División del nodo por Conciencia Ecológica

Table 3.13: Datos cons.eco.baja.

ID	Salario	¿Compra?
3	Bajo	No
5	Alto	Sí
6	Medio	No

Table 3.14: Datos cons.eco.alta.

ID	Salario	¿Compra?
1	Alto	Sí
2	Medio	Sí
4	Medio	Sí
7	Bajo	No
8	Medio	Sí
9	Bajo	Sí
10	Alto	Sí

Sobre cada tabla de datos se aplica el mismo procedimiento, para seguir ramificando.

3.2 Algoritmo CART

Este algoritmo, por el contrario, se utiliza preferentemente cuando la mayoría de los datos son de naturaleza numérica y el objetivo principal radica en la predicción.

Dicho procedimiento se divide en dos variantes, dependiendo de si el propósito es generar un árbol de clasificación o un árbol de regresión. Tal como se expuso en el Capítulo 2, la diferencia fundamental reside en que, en el árbol de clasificación, la variable objetivo pertenece a un conjunto finito de categorías, mientras que en los modelos de regresión esta puede tomar valores dentro de un rango continuo (conjunto infinito).

Como se ha mencionado anteriormente, el fundamento del algoritmo consiste en identificar la variable y el valor de corte que generen la partición óptima de los datos en el plano, siguiendo el esquema que se detalla a continuación:

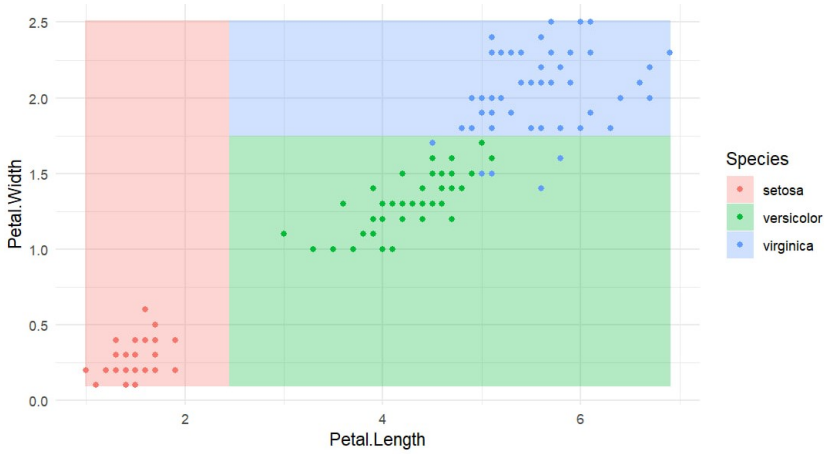


Figure 3.1: División del plano (clasificación)

El conjunto de datos utilizado corresponde a la base de datos Iris, la cual contiene registros de 150 ejemplares. Para este análisis, se han seleccionado únicamente las variables longitud y anchura del pétalo, con el fin de clasificar las especies en función de estas dos dimensiones.

En la Figura 3.1[1] se observa que las flores de tipo Setosa quedan clasificadas perfectamente mediante un único corte. Sin embargo, existe un solapamiento parcial entre las especies Virginica y Versicolor.

A continuación, se presenta otro ejemplo de aplicación.

En este ejemplo particular, se analiza el consumo de combustible (MPG) en función de la potencia (hp) y el peso (wt). Al tratarse de un árbol de regresión, se observa que el plano se divide en secciones, como se muestra en la Figura 3.2[2]. Sin embargo, no se realiza una clasificación, sino que se asigna un valor predicho constante para todas las observaciones contenidas en dicha área.

El algoritmo busca, por tanto, el punto de corte óptimo que maximice la pureza de los nodos o, en su defecto, minimice el número de clasificaciones erróneas.

[1]El código de la figura 3.1 se dejará en el Anexo 7
[2]El código de la figura 3.2 se dejará en el Anexo 7

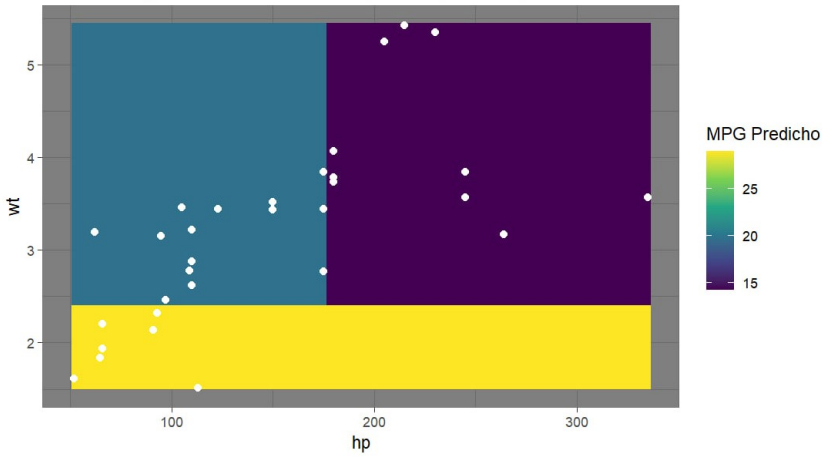


Figure 3.2: División del plano (Regresión)

A continuación, se describe la metodología para hallar el corte óptimo, analizando de manera independiente los árboles de clasificación y los de regresión. Una vez establecido el procedimiento, se procede a la definición de los conceptos matemáticos que sustentan el modelo.

3.2.1 Árboles de Regresión

Se asume la existencia de p variables predictoras $X_1, \ldots, X_p$. Para cada valor s, se genera una partición del plano (X_j, Y) definida por las condiciones $X_j \leq s$ y $X_j > s$. En cada partición, la predicción se realiza mediante la media de los puntos pertenecientes a dicha región, calculando posteriormente el error cuadrático respecto a dicha media.

Este procedimiento se ejecuta para cada valor s y para cada variable predictora; posteriormente, se selecciona la variable y el valor que generen el mínimo error, dado que el objetivo es predecir con la mayor precisión posible.

Se considera a X_j como la variable que genera la partición y a s como el valor de corte. Una vez establecida la división, se extraen de la base de datos inicial las observaciones que verifican $X_j \leq s$ y, por otro lado, aquellas que cumplen $X_j > s$. De este modo, se crean dos regiones sobre las cuales se aplica de nuevo el procedimiento anterior.

Si se procede de manera iterativa, se obtiene la estructura ramificada característica del árbol y, por ende, las particiones de la totalidad del plano. El algoritmo finaliza cuando se determina que el error no puede ser mejorado significativamente en ninguna de las regiones existentes, de modo que las regiones finales pasan a representar los nodos hoja del modelo.

A continuación, se presenta la formalización matemática del proceso. Se asume la existencia de M regiones $R_1, R_2, \ldots, R_M$. Para cada región R_m, se asigna un valor constante c_m, el cual se emplea para predecir el valor de las observaciones pertenecientes a dicha región:

$$f(x_i) = \sum_{m=1}^{M} c_m I\{x_i \in R_m\}.$$

Donde x_i representa el vector de variables predictoras del individuo i cuya respuesta se desea predecir, y la función indicatriz $I\{x_i \in R_m\}$ se define de la siguiente manera:

$$I\{x_i \in R_m\} = \begin{cases} 1 & \text{si } x_i \in R_m, \\ 0 & \text{si } x_i \notin R_m. \end{cases}$$

Dado que el objetivo consiste en minimizar la suma de los cuadrados de los residuos, $\sum(y_i - f(x_i))^2$, se determina que el estimador óptimo $\hat{c}_m$ corresponde a la media aritmética de las observaciones y_i que pertenecen a la región R_m. Por lo tanto, se define como:

$$\hat{c}_m = ave(y_i : x_i \in R_m).$$

Tal como se ha expuesto con anterioridad, el propósito es dividir el plano en regiones. Al considerar la variable X_j como la variable de separación y el punto s como el valor de corte, las regiones se definen de la siguiente manera:

$$R_1(j, s) = \{X | X_j \leq s\} \quad \text{y} \quad R_2(j, s) = \{X | X_j > s\}.$$

Para obtener la variable X_j y el punto de corte s óptimos, se minimiza la suma de los errores cuadráticos de las regiones $R_1(j, s)$ y $R_2(j, s)$ para cada combinación de (j, s). Por lo tanto, el problema de optimización se plantea de la siguiente manera:

$$\min_{j,s} \left[\min_{c_1} \sum_{x_i \in R_1(j,s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j,s)} (y_i - c_2)^2 \right].$$

Por lo tanto, para un par (j, s) se tiene que los valores c_1, c_2 vienen dados por:

$$c_1 = ave(y_i | x_i \in R_1(j, s)) \quad \text{y} \quad c_2 = ave(y_i | x_i \in R_2(j, s)).$$

Este algoritmo presenta un carácter iterativo y finaliza cuando el error en las regiones alcanza un valor mínimo. No obstante, se considera que la estructura del árbol puede optimizarse mediante la eliminación de ciertas ramas, ya que en ocasiones ocurre un sobreajuste (*overfitting*). Este fenómeno implica que el modelo memoriza los datos de entrenamiento en lugar de identificar los patrones subyacentes, lo que derivaría en errores predictivos al procesar nuevos datos.

Con este propósito, se emplea el método de la poda (*pruning*) para mejorar la capacidad de generalización del modelo, procedimiento que se analiza detalladamente en las secciones posteriores de este capítulo.

Finalmente, el proceso se puede resumir en el algoritmo 3:

Algorithm 3 Algoritmo CART de regresión

1. Con todo el conjunto de datos se calcula

$$\min_{j,s}\left[\min_{c_1}\sum_{x_i\in R_1(j,s)}(y_i-c_1)^2+\min_{c_2}\sum_{x_i\in R_2(j,s)}(y_i-c_2)^2\right],$$

   donde $c_1 = ave(y_i|x_i \in R_1(j,s))$ y $c_2 = ave(y_i|x_i \in R_2(j,s))$.

   Una vez que se haya calculado la primera división, se dividen los datos en dos: los que pertenecen a $R_1(j,s) = \{X|X_j \leq s\}$ y los que pertenecen a $R_2(j,s) = \{X|X_j > s\}$, donde en cada región se vuelve a realizar el mismo procedimiento.

2. Una vez determinadas las regiones, se puede clasificar cualquier nuevo individuo mediante la expresión:

$$f(x_i) = \sum_{m=1}^{M} \hat{c}_m I\{x_i \in R_m\},$$

   donde $\hat{c}_m = ave(y_i : x_i \in R_m)$.

3.2.2 Árboles de Clasificación

La premisa fundamental es análoga a la de los árboles de regresión; no obstante, la distinción principal radica en la función de pérdida utilizada. En este contexto, en lugar de minimizar el error cuadrático medio, se emplean el índice de Gini y la entropía, los cuales operan como métricas de impureza y desorden, respectivamente.

Para formalizar el modelo, se procede a definir las variables pertinentes. Considerando un nodo m del árbol, el cual representa una región específica R_m que contiene un número N_m de observaciones, se establece la siguiente notación:

$$\hat{p}_{mk} = \frac{1}{N_m} \sum_{x_i \in R_m} I(y_i = k),$$

a la probabilidad de que la variable y_i tome el valor k (k es una clase de la variable objetivo).

Existen diversos procedimientos para cuantificar la impureza de un nodo. El método de mayor implementación es el índice de Gini, descrito en [3], el cual se define mediante la siguiente expresión:

$$Q_m = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk}) = 1 - \sum_{k=1}^{K} \hat{p}_{mk}^2,$$

donde K es el número de total de clases que tiene la variable objetivo.

El cálculo del índice de Gini permite evaluar la impureza de un nodo o región de forma individual. Sin embargo, el proceso de construcción del árbol requiere identificar la variable y el punto de corte óptimos. Por consiguiente, se busca aquella división que minimice la impureza combinada de las regiones resultantes.

$$Q_{pond} = \frac{N_{m_1}}{N} Q_{m_1} + \frac{N_{m_2}}{N} Q_{m_2},$$

donde N es la suma del número de observaciones entre ambas regiones.

Como alternativa a la minimización de la impureza ponderada (Q_{pond}), es posible plantear el problema desde la maximización de la pureza. Esta equivalencia matemática radica en que minimizar Q_{pond} es análogo a maximizar la ganancia de información o reducción de la impureza, expresada mediante la siguiente relación:

$$\Delta Q_m = Q_m - Q_{pond}.$$

Maximizando, se llega a que:

$$\max \Delta Q_m = \max \left[Q_m - Q_{pond} \right] = \max \left[Q_m - \left(\frac{N_{m_1}}{N} Q_{m_1} + \frac{N_{m_2}}{N} Q_{m_2} \right) \right].$$

En consecuencia, al reducir la impureza de los nodos hijos, el valor de Q_{pond} disminuye, lo que resulta en un incremento de ΔQ_m. Dado que Q_m representa la impureza del nodo original antes de la partición, una menor impureza residual en los nodos resultantes implica una mayor ganancia en la homogeneidad global.

Un método alternativo para la selección de la variable de división es el uso de la entropía. Este concepto permite cuantificar la incertidumbre o el desorden intrínseco en un conjunto de datos, utilizándose en este contexto para evaluar el grado de dispersión de las clases dentro de un nodo.

Sea un nodo m con K categorías, la entropía mide la impureza o heterogeneidad de un conjunto —en este caso, el nodo m— respecto a una variable de clase. Para un problema de clasificación con K clases, se define como:

$$H_m = -\sum_{i=1}^{K} p_i \log_2(p_i),$$

donde p_i es la proporción de ejemplos de la clase i en m. Toma valor 0 cuando el conjunto es puro (todos los ejemplos pertenecen a la misma clase) y valor máximo $log_2(K)$ cuando las clases están equiprobablemente distribuidas entre todas las clases, es decir, $p_i = \frac{1}{K}$ para todo i.

En consecuencia, se establece que un valor elevado de entropía indica una alta heterogeneidad o mezcla entre las clases presentes en el nodo. Por el contrario, una entropía baja es indicativa de un nodo con mayor pureza, donde la predominancia de una única clase es significativa.

Respecto a la base del logaritmo empleada en la definición, es habitual el uso del logaritmo natural; no obstante, la aplicación del logaritmo en base 2 no altera el orden jerárquico ni los resultados en el proceso de selección de la variable de división. Esto se debe a que el cambio de base solo implica un factor de escala constante, manteniendo así la consistencia en el rango de elección de las particiones.

El propósito de la entropía consiste en identificar la variable y el punto de corte que minimicen el desorden o la mezcla de clases en las regiones resultantes. Este proceso se fundamenta en el cálculo de la ganancia de información, la cual se define como la diferencia entre la entropía inicial del nodo y la entropía ponderada tras la partición. El algoritmo selecciona, por tanto, la división que maximiza dicha diferencia, siguiendo una lógica análoga a la aplicada con el índice de Gini. La entropía ponderada se define como:

$$H_{pond} = \frac{N_{m_1}}{N} H_{m_1} + \frac{N_{m_2}}{N} H_{m_2},$$

donde N es el número de observaciones totales entre los dos nodos, y m_1, m_2 son las dos regiones en las que se divide m.

Formalmente, la ganancia de información (ΔH_m) se expresa como:

$$\Delta H_m = H_m - H_{pond}.$$

Maximizando la ganancia de información:

$$\max \Delta H_m = \max \left[H_m - H_{pond} \right] = \max \left[H_m - \left(\frac{N_{m_1}}{N} H_{m_1} + \frac{N_{m_2}}{N} H_{m_2} \right) \right].$$

Al maximizar esta diferencia, se garantiza que la división elegida sea aquella que reduzca de forma más drástica la incertidumbre, logrando así que los nodos descendientes sean lo más puros posible.

Para los árboles de clasificación, el procedimiento puede sintetizarse en el algoritmo 4:

Algorithm 4 Algoritmo CART de clasificación

1. Con todos los datos iniciales se calcula el mejor corte usando el índice de gini o la entropía.

   - **Índice Gini**:
   $$Q_{pond} = \frac{N_{m_1}}{N} Q_{m_1}(T) + \frac{N_{m_2}}{N} Q_{m_2}(T).$$
   - **Entropía**:
   $$H_{pond} = \frac{N_{m_1}}{N} H_{m_1} + \frac{N_{m_2}}{N} H_{m_2}.$$

   Una vez obtenida la partición inicial, los datos se segmentan y se procede a aplicar el mismo procedimiento de forma recursiva en cada una de las regiones resultantes.

2. Finalmente, se realiza este método de manera iterativa.

En relación con la capacidad predictiva de ambas métricas, la evidencia empírica sugiere que la elección entre el índice de Gini y la entropía suele tener un impacto mínimo en la precisión final del modelo de clasificación. Generalmente, ambas medidas conducen

a la construcción de árboles con estructuras muy similares. El índice de Gini es generalmente preferido en aplicaciones donde la velocidad de cómputo es prioritaria, debido a su simplicidad algebraica, mientras que la entropía ofrece una medida de desorden basada en la teoría de la información que resulta útil cuando se busca una sensibilidad mayor ante la distribución de las clases.

Para ilustrar el funcionamiento del algoritmo, se presenta una iteración del método aplicada a un escenario práctico de clasificación.

El objetivo consiste en predecir el interés de un espectador por una nueva entrega de Star Wars basándose en dos variables predictoras: la afinidad previa por Star Trek y la edad del individuo. Para este ejercicio se empleará el índice de Gini, aunque, como se ha expuesto anteriormente, el procedimiento sería análogo utilizando la entropía.

A continuación, se detallan los datos de entrenamiento que conforman el conjunto de observaciones inicial:

Table 3.15: Datos árbol de clasificación

¿Le gustó Star Trek?	Edad	¿Le gustó Star Wars?
Sí	22	Sí
Sí	50	No
No	25	Sí
Sí	38	No
No	44	Sí
No	15	No
Sí	45	Sí

Para iniciar la construcción del árbol, es preciso determinar qué variable actuará como nodo raíz. Con este fin, se procede al cálculo del índice de Gini ponderado para cada una de las variables predictoras.

En este análisis, se define la variable de respuesta de la siguiente forma: $k = 1$ si el espectador muestra afinidad por Star Wars y $k = 2$ en caso contrario. A continuación, se calculan las proporciones $\hat{p}_{mk}$ para cada escenario.

Al segmentar los datos según esta variable, se originan dos regiones: Región 1 (R_1), espectadores a los que les gustó Star Trek ($m = 1$), y Región 2 (R_2), espectadores a los que no les gustó Star Trek ($m = 2$).

Análisis para la región 1, R_1:

- $\hat{p}_{11} = \frac{2}{4} = \frac{1}{2}$ donde $m = 1, k = 1$ (hay 4 personas a las que le gustó Star Trek y, de esas, a 2 les gustó Star Wars).
- $\hat{p}_{12} = \frac{2}{4} = \frac{1}{2}$ donde $m = 1, k = 2$ (hay 4 personas a las que le gustó Star Trek y, de esas, a 2 no les gustó Star Wars).

Luego, el índice de Gini para esa región R_1 viene dado por:

$$Q_1(T) = \sum_{k=1}^{2} \hat{p}_{1k}(1-\hat{p}_{1k}) = \hat{p}_{11}(1-\hat{p}_{11})+\hat{p}_{12}(1-\hat{p}_{12}) = \frac{1}{2}\left(1-\frac{1}{2}\right)+\frac{1}{2}\left(1-\frac{1}{2}\right) = \frac{1}{2}.$$

Para la región 2, R_2:

- $\hat{p}_{21} = \frac{2}{3}$ donde $m = 2, k = 1$ (hay 3 personas a las que no les gustó Star Trek y, de esas, a 2 les gustó Star Wars).
- $\hat{p}_{22} = \frac{1}{3}$ donde $m = 2, k = 2$ (hay 3 personas a las que no les gustó Star Trek y, de esas, a 1 no les gustó Star Wars).

Luego, el índice de Gini para esa región R_2 viene dado por:

$$Q_2(T) = \sum_{k=1}^{2} \hat{p}_{2k}(1-\hat{p}_{2k}) = \hat{p}_{21}(1-\hat{p}_{21})+\hat{p}_{22}(1-\hat{p}_{22}) = \frac{2}{3}\left(1-\frac{2}{3}\right)+\frac{1}{3}\left(1-\frac{1}{3}\right) = \frac{4}{9}.$$

Luego, para la variable ¿Les gustó Star Trek? tenemos un índice de ponderación:

$$Q_{pond} = \frac{N_1}{N}Q_1(T) + \frac{N_2}{N}Q_2(T) = \frac{4}{7}\frac{1}{2} + \frac{3}{7}\frac{4}{9} = \frac{10}{21} = 0.476.$$

Luego, el índice de Gini ponderado es 0.476.

Para evaluar la capacidad predictiva de la variable Edad (X_2), es necesario realizar un tratamiento específico dado su carácter numérico continuo. A diferencia de las variables categóricas, las variables cuantitativas requieren la identificación de un punto de corte óptimo mediante un proceso de discretización.

Primero se organizan las observaciones de menor a mayor en función de la edad, para posteriormente determinar los valores medios entre cada par de edades adyacentes. Estos valores actuarán como candidatos para el punto de corte. A continuación, se presenta la tabla de datos reordenada.

Table 3.16: Ordenación

Edad
15
22
25
38
44
45
50

Table 3.17: Media

Edad
18.5
23.5
31.5
41
44.5
47.5

Siguiendo el procedimiento descrito, se procede a evaluar cada uno de los puntos de corte candidatos para la variable Edad. En cada caso, se segmentará el conjunto de datos en dos regiones y se calcularán las proporciones $\hat{p}_{mk}$ basándose en la distribución de la variable de respuesta en los datos de entrenamiento.

Al establecer el umbral en 18.5, se definen las siguientes regiones: R_1 Si Edad≤ 18.5 y R_2 Si Edad> 18.5, y para calcular $\hat{p}_{mk}$ donde se usarán los datos iniciales. Analizando la región R_1 tenemos:

- $\hat{p}_{11} = 0$ donde $m = 1, k = 1$ (la única persona con Edad≤ 18.5 no le gusta Star Wars).
- $\hat{p}_{12} = 1$ donde $m = 1, k = 2$ (la única persona con Edad≤ 18.5 no le gusta Star Wars).

Luego, el índice de Gini para esa región R_1 viene dado por:

$$Q_1(T) = \sum_{k=1}^{2} \hat{p}_{1k}(1 - \hat{p}_{1k}) = \hat{p}_{11}(1 - \hat{p}_{11}) + \hat{p}_{12}(1 - \hat{p}_{12}) = 0.$$

Para la otra región R_2 se tendrá:

- $\hat{p}_{21} = \frac{4}{6} = \frac{2}{3}$ donde $m = 2, k = 1$ (de las 6 personas, a 4 les gustó Star Wars).
- $\hat{p}_{22} = \frac{2}{6} = \frac{1}{3}$ donde $m = 2, k = 2$ (de las 6 personas, a 2 no les gustó Star Wars).

Luego, el índice de Gini para esa región R_2 viene dado por:

$$Q_2(T) = \sum_{k=1}^{2} \hat{p}_{2k}(1-\hat{p}_{2k}) = \hat{p}_{21}(1-\hat{p}_{21})+\hat{p}_{22}(1-\hat{p}_{22}) = \frac{2}{3}\left(1-\frac{2}{3}\right)+\frac{1}{3}\left(1-\frac{1}{3}\right) = \frac{4}{9}.$$

Luego, para la variable Edad tomando como corte la edad ≤ 18.5 se tiene un índice de ponderación:

$$Q_{pond} = \frac{N_1}{N}Q_1(T) + \frac{N_2}{N}Q_2(T) = \frac{1}{7}0 + \frac{6}{7}\frac{4}{9} = \frac{8}{21} = 0.380.$$

Al comparar este resultado con el obtenido previamente para la variable Star Trek ($Q_{pond} = 0.476$), se observa que la variable categórica genera, en este primer análisis, una mayor reducción de la impureza. No obstante, para concluir cuál será el nodo raíz, el algoritmo debe completar el cálculo de todos los puntos de corte restantes de la variable Edad ($23.5, 31.5, 41, 44.5$ y 47.5) y seleccionar el valor mínimo absoluto entre ambas variables.

Ahora las dos regiones son R_1 si Edad≤ 23.5 y R_2 si Edad> 23.5, y se calculará $\hat{p}_{mk}$. Para la región R_1 tenemos:

- $\hat{p}_{11} = \frac{1}{2}$ donde $m = 1, k = 1$ (de las dos personas con Edades≤ 23.5, a una le gustó Star Wars).
- $\hat{p}_{12} = \frac{1}{2}$ donde $m = 1, k = 2$ (de las dos personas con Edades≤ 23.5, a una no le gustó Star Wars).

Luego, el índice de Gini para esa región R_1 viene dado por:

$$Q_1(T) = \sum_{k=1}^{2} \hat{p}_{1k}(1-\hat{p}_{1k}) = \hat{p}_{11}(1-\hat{p}_{11}) + \hat{p}_{12}(1-\hat{p}_{12}) = \frac{1}{2}\frac{1}{2} + \frac{1}{2}\frac{1}{2} = \frac{1}{2}.$$

Para la otra región R_2 se obtendrá:

- $\hat{p}_{21} = \frac{3}{5}$ donde $m = 2, k = 1$ (de las 5 personas con edades> 23.5, a tres les gustó Star Wars).
- $\hat{p}_{22} = \frac{2}{5}$ donde $m = 2, k = 2$ (de las 5 personas con edades> 23.5, a dos no les gustó Star Wars).

Luego, el índice de Gini para esa región R_2 viene dado por:

$$Q_2(T) = \sum_{k=1}^{2} \hat{p}_{2k}(1-\hat{p}_{2k}) = \hat{p}_{21}(1-\hat{p}_{21})+\hat{p}_{22}(1-\hat{p}_{22}) = \frac{3}{5}\left(1-\frac{3}{5}\right)+\frac{2}{5}\left(1-\frac{2}{5}\right) = \frac{12}{25}.$$

Luego, para la variable Edad≤ 23.5 se obtiene un índice de ponderación:

$$Q_{pond} = \frac{N_1}{N}Q_1(T) + \frac{N_2}{N}Q_2(T) = \frac{2}{7}\frac{1}{2} + \frac{5}{7}\frac{12}{25} = \frac{17}{35} = 0.487.$$

Este es el índice de Gini si decidimos usar la variable Edad≤ 23.5 como raíz.

A continuación, se repite el mismo proceso con el resto: Edad≤ 31.5, Edad≤ 41, Edad≤ 44.5 y Edad≤ 47.5.

Table 3.18: Comparativa de índices de Gini ponderados para la variable Edad.

Punto de corte	**Gini Ponderado**
¿Le gustó Star Trek?	0.4762
Edad ≤ 18.5	**0.3810**
Edad ≤ 23.5	0.4857
Edad ≤ 31.5	0.4762
Edad ≤ 41	0.4762
Edad ≤ 44.5	0.4857
Edad ≤ 47.5	**0.3810**

El conjunto presenta un empate entre los cortes **Edad** ≤ 18.5 y **Edad** ≤ 47.5. La implementación rpart de R recorre los candidatos en orden creciente y escoge el primero que alcanza el mínimo. Bajo esta regla, el nodo raíz queda definido por **Edad** ≤ 18.5.

Los dos subconjuntos de datos con los cortes Edad ≤ 18.5 y Edad > 18.5 quedan reflejados en las tablas 3.19 y 3.20:

Table 3.19: Datos Edad ≤ 18.5

¿Le gustó Star Trek?	**Edad**	**¿Le gustó Star Wars?**
No	15	No

Finalmente, habría que repetir el mismo razonamiento hasta finalizar. Para el caso Edad ≤ 18.5 no haría falta seguir dividiendo, ya que solo hay un individuo; se continúa dividiendo solo la hoja Edad > 18.5.

Table 3.20: Datos Edad > 18.5

¿Le gustó Star Trek?	Edad	¿Le gustó Star Wars?
Sí	50	No
Sí	38	No
No	44	Sí
Sí	45	Sí
Sí	22	Sí
No	25	Sí

3.2.3 Criterios de parada y control del sobreajuste en CART

A diferencia de una explicación puramente cualitativa basada en la minimización del error, el crecimiento de un árbol CART está regulado por reglas de parada predefinidas. Estos hiperparámetros son críticos para equilibrar el compromiso entre sesgo y varianza, evitando que el modelo se ajuste excesivamente al ruido de los datos de entrenamiento.

De acuerdo con lo descrito en [4], los criterios de parada más relevantes son:

- **Tamaño mínimo de nodo (n_{min}):** establece el número mínimo de observaciones requeridas en un nodo para intentar una división. Un n_{min} pequeño permite árboles más profundos y complejos, aumentando el riesgo de sobreajuste.
- **Profundidad máxima (d_{max}):** limita el número máximo de niveles del árbol. Una profundidad excesiva permite al algoritmo capturar interacciones muy específicas de la muestra, lo que degrada la capacidad de generalización.
- **Umbral de mejora de impureza (ΔI_{min}):** una división solo se lleva a cabo si la reducción en la impureza (ya sea Gini o Entropía) supera un valor crítico ΔI. Si ninguna división posible mejora el criterio por encima de este umbral, el nodo se convierte en terminal.
- **Umbral de mejora del error:** el árbol deja de dividirse si el error en ese nodo es menor que un valor crítico (un umbral cercano a cero). Si el error ya es muy pequeño, significa que todas las observaciones de ese nodo tienen valores muy parecidos; el nodo ya es lo bastante preciso y no necesita más divisiones.

Un árbol que crece sin restricciones hasta que cada hoja sea pura tendrá un sesgo nulo, pero una varianza extremadamente alta. La formalización de estos criterios actúa como

una forma de *pre-poda*, limitando la complejidad del modelo *a priori*. Como señala [4], la estrategia óptima suele consistir en permitir un crecimiento amplio y posteriormente aplicar una poda (*pruning*) basada en la complejidad del costo.

3.3 Poda

Como se ha mencionado anteriormente, el algoritmo puede ajustarse de forma excesiva a los datos de entrenamiento, comprometiendo su capacidad de predicción ante nuevos datos. Este fenómeno puede identificarse comparando el error cometido en el conjunto de entrenamiento con el error estimado mediante validación cruzada: si el error de entrenamiento es considerablemente inferior al error de validación cruzada, el árbol está sobreajustado. A fin de mitigar este problema, se realiza la poda por complejidad de coste (*Cost Complexity Pruning*). Este método busca establecer un equilibrio entre la complejidad del árbol (determinada por el número de nodos hoja) y su precisión (reflejada en la tasa de error).

Para este propósito, se emplea la siguiente expresión:

$$C_\alpha(T) = R(T) + \alpha|T|, \tag{3.7}$$

donde $|T|$ es el número de nodos hoja del árbol T. El α actúa como penalizador por tener muchas hojas, y $R(T)$ es la tasa de error.

$R(T)$ también puede escribirse como:

$$R(T) = \sum_{m=1}^{|T|} N_m Q_m(T),$$

donde las variables son:

$$N_m = \#\{x_i \in R_m\},$$

$$\hat{c}_m = \frac{1}{N_m} \sum_{x_i \in R_m} y_i,$$

$$Q_m(T) = \frac{1}{N_m} \sum_{x_i \in R_m} (y_i - \hat{c}_m)^2.$$

La ecuación 3.7 se usará para saber qué subárboles son los candidatos para ser el predictor.

Se define un subárbol $T \subset T_0$ como cualquier árbol que se obtenga por la poda de T_0, es decir, colapsando un número determinado de nodos internos.

La poda se realiza identificando los nodos internos cuya eliminación no incremente el coste total. Por ejemplo, si al podar un nodo se reduce el número de nodos terminales de $|T|$ a $|T'|$, el error cuadrático medio pasa de $R(T)$ a $R(T')$, y no hay incremento del coste total, es decir, $C_\alpha(T) \geq C_\alpha(T')$, entonces:

$$0 \leq C_\alpha(T) - C_\alpha(T') = R(T) - R(T') + \alpha(|T| - |T'|)$$

O, lo que es lo mismo:

$$\alpha = \frac{R(T') - R(T)}{|T| - |T'|},$$

es el mínimo necesario para justificar la poda de esa rama.

Para determinar el valor de α óptimo, se emplea el método de validación cruzada, explicado en el algoritmo 5.

En la práctica, el error de validación cruzada se expresa de forma normalizada respecto al error del árbol raíz, denominándose xerror, de modo que un valor igual a 1 indica que el árbol no mejora respecto a predecir siempre la clase mayoritaria.

No obstante, minimizar el error de validación cruzada no siempre conduce al modelo más adecuado. Una alternativa ampliamente utilizada es la regla 1-SE (*one standard error rule*), que selecciona el árbol más simple cuyo error de validación cruzada no supere: $\text{xerror}_{\min} + \text{xstd}_{\min}$, donde $\text{xerror}_{\min}$ es el mínimo error obtenido y $\text{xstd}_{\min}$ es su desviación estándar asociada. La intuición detrás de esta regla es que, si dos árboles tienen un rendimiento estadísticamente similar, es preferible el más simple, por su mayor interpretabilidad y menor riesgo de sobreajuste.

Algorithm 5 Poda por complejidad de costo y validación cruzada

1. Primero se crea la secuencia de subárboles:
   - Con el conjunto de datos completo se construye el árbol completo.
   - Para cada nodo de decisión se calcula el α que se corresponde con la eliminación de esa rama.
   - Una vez calculados los valores de α asociados a cada nodo de decisión del árbol actual, se escoge el menor de ellos (α_1). Al podar el nodo correspondiente a dicho mínimo, se obtiene el subárbol T_1.
   - Una vez tengamos T_1, volvemos a calcular los α asociados a cada nodo de decisión pero esta vez del árbol T_1, repitiendo el mismo proceso.
2. Una vez obtenida la secuencia $T_m \subset ... \subset T_2 \subset T_1 \subset T_0$, escogemos el mejor subárbol mediante validación cruzada:
   - Los datos se dividirán en un número k de subconjuntos (habitualmente 5 o 10). El modelo se entrena con $k-1$ subconjuntos, mientras que el restante se reserva para la fase de pruebas.
   - Para cada valor de α de la secuencia, se entrena un árbol con los $k-1$ folds, se le aplica la poda correspondiente a ese α y se calcula su error en el fold de validación.
   - Este proceso se repite de manera cíclica, rotando el subconjunto de prueba mediante validación cruzada, calculando el error de predicción en cada subárbol.
   - Finalmente se promedian los resultados obtenidos de la validación cruzada y se selecciona el α^* que minimice el error de predicción. El árbol que se escogerá será el que esté asociado al α^*

CAPITULO

4

Bosques aleatorios

Como se mencionó en el Capítulo 2, los *Random Forests*, propuestos por Breiman en [5], servirán para mejorar el *overfitting* usando el algoritmo del *Bagging* mencionado en el mismo capítulo.

La idea del algoritmo *Random Forest* es simple. Primero crearemos B subconjuntos de datos de la misma dimensión que el conjunto de datos inicial, supóngase que sea n. Luego se crean B árboles para posteriormente hacer una ponderación/media de los resultados obtenidos por todos los árboles. El proceso completo es conocido como *Bagging*. También es importante saber que para la construcción de cada árbol se tomará un subconjunto aleatorio m de las p covariables para realizar la división del nodo.

La creación de los m subconjuntos de datos se basa en el método *Bootstrap*, que consiste en que cada subconjunto de datos contiene n filas del conjunto inicial escogidas de manera aleatoria. Por lo tanto, en cada subconjunto de datos habrá datos repetidos y datos que no aparecerán en los subconjuntos. A estos se les denomina *Out-Of-Bag*.

Al aplicar el modelo a nuevas observaciones, supóngase y, el dato entrará en el bosque, donde cada árbol generará un resultado. A estos resultados se les aplicará el método de *Aggregation*, que consiste en hacer la media, si los árboles son de regresión, o la ponderación, si los árboles son de clasificación.

El algoritmo también calculará un error conocido como Error OOB. Para calcular este error, usará los valores *Out-Of-Bag* de la siguiente manera.

Supongamos que x es un valor *Out-Of-Bag*, entonces, cuando entre al bosque, será evaluado por los árboles que no hayan usado x para entrenar, para posteriormente realizar la ponderación/media de los árboles.

En este caso no hace falta dividir los datos en datos de entrenamiento y de testeo, ya que el OOB Error sirve para sacar conclusiones sobre cómo predice el modelo.

Este procedimiento queda resumido en el algoritmo 6, donde B denota el número de árboles del bosque y n el tamaño de nuestra muestra. Además, se supondrá a lo largo del capítulo que se tiene p variables predictoras (covariables).

Algorithm 6 Procedimiento de construcción de un Bosque Aleatorio (*Random Forest*)

1. Desde $b = 1$ a B haremos el siguiente proceso:
   - Recogemos una muestra aleatoria de los datos (*Bootstrap*), denotada Z^*, de tamaño n del conjunto de datos inicial.
   - Para ese subconjunto de datos crearemos un árbol de decisión siguiendo los siguientes pasos hasta llegar tamaño del nodo n_{min} o la profundidad del árbol m_{depth} deseada:
     - Seleccionamos un número m de las p variables predictoras de manera aleatoria.
     - Escogemos la variable que nos proporcione la mejor división dentro de esas m.
     - Dividimos ese nodo en dos nodos.
2. Una vez finalizado el proceso para cada árbol, tendremos el conjunto de árboles $\{T_b\}_1^B$, donde $1 \leq b \leq B$.

Para realizar una predicción de un nuevo elemento x haremos una media o ponderación dependiendo de si es un árbol de clasificación o regresión.
Regresión: $\hat{f}^B(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$ (Media de los resultados de cada árbol).
Clasificación: Si denotamos $\hat{C}_b(x)$ a la predicción del árbol b-ésimo, tendremos que la predicción del bosque es $\hat{C}^B(x) =$ Votación mayoritaria$\{\hat{C}_b(x)\}_1^B$ (Ponderación de los resultados).

Una vez explicado el algoritmo, se verá el razonamiento de este procedimiento.

Es lógico pensar que los árboles con mucha profundidad tienen un bajo sesgo, ya que se ajustan muy bien a los datos, pero presentan una alta varianza, ya que si se cambia un dato, el árbol resulta totalmente distinto.

Además, se observa que los árboles de un bosque aleatorio son i.d. (idénticamente distribuidos); esto es lo mismo que decir que nacen del mismo proceso, es decir, se crean de la misma manera: se usa el mismo conjunto de datos, el mismo criterio de parada, la misma métrica de impureza y se elige el mismo número de variables para desplegar en cada nodo. Por lo tanto, se verifica lo siguiente:

$$E\left(\frac{1}{B}\sum_{b=1}^{B}T_b(x)\right)=\frac{1}{B}\sum_{b=1}^{B}E[T_b(x)]=\frac{1}{B}B\mu,$$

donde μ es la esperanza del árbol (como son i.d., tenemos que la esperanza es la misma).

Esto significa que el sesgo del bosque usando el método *Bagging* es el mismo que el que se obtiene de los árboles individuales, luego la única manera de mejorar el modelo es reduciendo la varianza.

Proposición 4.0.1 (Propiedades de la Varianza). Sea X una variable aleatoria e $X_1, X_2, ..., X_n$ variables aleatorias independientes e idénticamente distribuidas (i.i.d.), entonces se verifican las siguientes propiedades:

$$\text{Var}(aX)=a^2\text{Var}(X), \tag{4.1}$$

$$\text{Var}\left(\sum_{i=1}^{n}X_i\right)=\sum_{i=1}^{n}\text{Var}(X_i)=\sum_{i=1}^{n}\sigma^2=n\sigma^2. \tag{4.2}$$

Luego, si los árboles que forman parte del bosque tienen varianza σ^2 y son independientes, se tiene que la varianza del promedio es la siguiente:

$$\text{Var}\left(\frac{1}{B}\sum_{b=1}^{B}T_b(x)\right)=\frac{1}{B^2}\sum_{b=1}^{B}\text{Var}(T_b(x))=\frac{1}{B^2}B\sigma^2=\frac{1}{B}\sigma^2.$$

Sin embargo, si los árboles solo son i.d., es decir, no son independientes, existe una correlación entre ellos. Por lo tanto, la varianza del bosque se calcularía como:

$$\rho\sigma^2+\frac{1-\rho}{B}\sigma^2,$$

cuya demostración se dejará en el Anexo 7.

Se puede observar que cuando el número de árboles (B) crece, el segundo término desaparece y la varianza del árbol dependerá de la correlación de los árboles. Luego, para reducir la varianza hay que reducir la correlación.

El método *Bootstrap*, descrito por Efron en [6], se basa en una idea clave: tratar la muestra que se tiene como si fuera una pequeña representación del mundo real. Al sacar

miles de muestras nuevas a partir de la original (usando remuestreo con reemplazo), se puede ver cómo cambia el resultado una y otra vez. Efron demostró que esta variación que se observa en la computadora es equivalente a la que se encontraría si se pudiera repetir el estudio con miles de grupos de pacientes diferentes. En los bosques aleatorios, esto se usa para que cada árbol aprenda de una versión distinta de los datos.

Se ha demostrado que si los árboles se parecen mucho, la correlación será alta, y además no tendría sentido promediarlos, ya que todos darían resultados parecidos.

Es por ello que, a la hora de formar los árboles, se toman m variables de las p covariables; esto es para descorrelar los árboles, es decir, para que no sean tan parecidos.

Una vez se tenga el bosque construido, el valor de la predicción viene dado por:

$$\hat{f}^B(x) = \frac{1}{B}\sum_{b=1}^{B} T(x;\Theta_b).$$

En este caso los árboles no se han denotado como T_b, sino que se han definido como $T(x;\Theta_b)$, donde Θ_b son los parámetros que caracterizan al árbol en términos de variables de división.

Conforme a lo indicado en [4] se recomienda usar:

- ***Clasificación***: $m = \sqrt{p}$ y como criterio de parada que el tamaño del último nodo sea $n_{min} = 1$.
- ***Regresión***: $m = \frac{p}{3}$ y como criterio de parada que el tamaño del último nodo sea $n_{min} = 5$.

Aunque, en la mayor parte de los casos, dependerá del tipo de problema.

Como se mencionó en el Capítulo 2, para realizar el testeo se usarán los datos de prueba, pero se podría realizar una permutación de los datos de una de las covariables. Esto puede llegar a dar información sobre la importancia de esa variable para el modelo. Este método es conocido como Importancia de Variables (*Variable Importance*).

En el Capítulo 3 se estudió que si la profundidad de un árbol (individual) era muy grande, se producía *overfitting*. Razonemos si en un bosque aleatorio puede producirse sobreajuste.

Es cierto que, al aumentar el número de árboles B, no puede producirse sobreajuste, ya que el promedio converge cuando $B \to \infty$, tal como se demuestra en [5]. No obstante, una elección incorrecta de hiperparámetros puede comprometer la capacidad de generalización

por otra vía: un valor de m excesivamente alto incrementa la correlación entre árboles, reduciendo el efecto de promediado del *bagging* sobre la varianza global (recordemos que $\mathrm{Var}(\hat{f}^B) = \rho\sigma^2 + (1-\rho)\sigma^2/B$, expresión que tiende a $\rho\sigma^2$ cuando ρ se aproxima a 1). El sobreajuste estricto de los árboles individuales, derivado de profundidades excesivas, no desaparece por agregación si los árboles son altamente correlacionados.

CAPITULO

5

Una Aplicación real

En este capítulo se detalla la implementación práctica de los modelos de Árboles de Decisión y Bosques Aleatorios (*Random Forest*) tratados en la memoria. El objetivo es aplicar estas técnicas de aprendizaje supervisado sobre un conjunto de datos clínico real para predecir el periodo de supervivencia de un paciente tratado de cáncer.

Para ello, se ha utilizado un dataset cuyos datos provienen del German Breast Cancer Study Group [7, 8], que recoge variables histopatológicas y clínicas de 686 pacientes con cáncer de mama. A través de este análisis, se pretende identificar los patrones de riesgo que determinan la supervivencia del paciente tras el tratamiento inicial.

Antes de proceder al entrenamiento del modelo, es necesario definir las características clínicas (predictores) que se utilizarán para determinar el periodo de supervivencia. El conjunto de datos original consta de 11 columnas, de las cuales se han seleccionado aquellas con valor clínico directo para el diagnóstico.

Las variables se pueden agrupar en tres categorías principales:

Variables clínicas:

- **Edad (*age*):** edad de la paciente en el momento del diagnóstico inicial; este es un valor numérico.
- **Estado menopáusico (*meno*):** variable binaria que indica si la paciente se encuentra en estado premenopáusico (0) o posmenopáusico (1).

Características del tumor y tratamiento:

- **Tamaño del tumor (*size*):** diámetro del tumor en milímetros.
- **Grado de agresividad (*grade*):** clasificación histológica del 1 al 3 que indica el nivel de agresividad y diferenciación de las células cancerosas.
- **Ganglios afectados (*nodes*):** número de ganglios linfáticos axilares positivos.

Marcadores hormonales y de tratamiento:

- **Receptores de progesterona (*pgr*) y estrógeno (*er*):** concentraciones medidas en fmol/l. Estos receptores determinan si el cáncer crece en respuesta a las hormonas.
- **Terapia hormonal (*hormon*):** indica si la paciente recibió tratamiento específico para bloquear hormonas (0: No, 1: Sí).

Y, por último, se tienen las variables status y rfstime:

- **Estado (*status*)**: es la variable que nos indica si el paciente ha recaído o no. Esta valdrá 0 si no hay recaída o 1 si hay recaída.
- **Tiempo (*rfstime*)**: esta es una variable de tiempo truncada, es decir, solo sabemos el estado del paciente en ese instante.

Además, el dataset tiene una variable más, que es *pid*, el indicador del paciente. Pero esta no se usará en el estudio, ya que no es información relevante.

Cabe destacar que, aunque en el Capítulo 3 se ha abordado el estudio teórico del algoritmo CHAID, la aplicación práctica desarrollada en esta memoria se ha centrado exclusivamente en el algoritmo CART y su extensión en *Random Forest*. Esta decisión metodológica se justifica por dos motivos fundamentales: por un lado, la naturaleza de los datos, ya que el conjunto de datos clínico utilizado (GBSG) cuenta con una presencia mayoritaria de variables numéricas y continuas; y, por otro, la reorientación del objetivo práctico, debido a que el análisis se ha reorientado hacia la maximización de la capacidad predictiva y la reducción del error de predicción.

Como **rfstime** es una variable truncada por la derecha, no se puede usar esa variable en los árboles de decisión abordados en el Capítulo 3, ya que no es un tiempo donde se curan o recaen. Lo único que se conoce es que si **status** es 0 y **rfstime** es, por ejemplo, 12, luego

a los 12 días estaba sano, pero no sabemos nada del paciente luego de ese tiempo. Esto puede deberse a que el estudio terminó o a que el paciente no quiso seguir participando en él. Sin embargo, si **status** es 1 y **rfstime** es 12, se sabe que a los doce días recayó.

En su lugar, se introducirá a los árboles de supervivencia (*Survival Trees*) y a los bosques de supervivencia (*Random Survival Forest*). Estos son modelos idénticos a los explicados anteriormente, con la única diferencia de que se usan cuando hay una variable temporal truncada, como es el caso de esta base de datos, además de que se usará otro criterio de división.

Antes de empezar con la programación se introducirá brevemente a estos tipos de árboles y bosques.

5.1 Árboles de Supervivencia

Para cada paciente i, se dispone del par (T_i, δ_i), donde:

$$T_i = \min(Y_i, C_i),$$

siendo Y_i el tiempo real hasta la recaída y C_i el tiempo de censura (final del estudio, pérdida de seguimiento, etc.). Cuando $\delta_i = 1$ se observa el evento; cuando $\delta_i = 0$ solo se sabe que el paciente seguía libre de recaída en el instante T_i. Ignorar la censura, por ejemplo tratando T_i como una variable continua y aplicando un árbol de regresión, introduce un sesgo grave: los pacientes con $\delta_i = 0$ aportarían tiempos artificialmente cortos que no corresponden a tiempos de evento reales. De ahí la necesidad de un marco específico.

5.1.1 Construcción del árbol de supervivencia

Un árbol de supervivencia conserva la estructura recursiva de CART, pero sustituye el criterio de impureza (Gini, entropía o MSE) por una medida que aprovecha la información temporal y la censura. El criterio más empleado, propuesto inicialmente por Ciampi en [9] y consolidado por LeBlanc y Crowley en [10], es el estadístico *log-rank*.

Dado un nodo con N individuos y una división candidata por la variable X_j en el punto s, se generan dos subgrupos $R_1(j, s) = \{X_j \leq s\}$ y $R_2(j, s) = \{X_j > s\}$. Sean $t_1 < t_2 < ... < t_m$ los instantes ordenados en los que ocurre al menos un evento. Definiremos d_{kl} los eventos observados en la región $k \in \{1, 2\}$ en el instante t_l, n_{kl} los individuos en

riesgo, $d_l = d_{1l} + d_{2l}$ y $n_l = n_{1l} + n_{2l}$. El estadístico log-rank se define como:

$$L(j,s) = \frac{\sum_l \left(d_{1,l} - n_{1,l}\frac{d_l}{n_l} \right)}{\sqrt{\sum_l \frac{n_{1,l}}{n_l}\left(1 - \frac{n_{1,l}}{n_l}\right)\left(\frac{n_l - d_l}{n_l - 1}\right) d_l}}.$$

La partición óptima es la que maximiza $|L(j,s)|$, es decir, la que produce dos subgrupos con curvas de supervivencia lo más diferentes posible. La intuición es paralela a CART: en lugar de minimizar impureza, se maximiza la separación entre las dinámicas temporales de los nodos hijos.

5.1.2 Predicción en los nodos terminales

A diferencia de un árbol de clasificación, donde un nodo hoja predice una clase, o de uno de regresión, donde predice un valor constante, un árbol de supervivencia asigna a cada hoja h una función de supervivencia estimada $\widehat{S}_h(t)$, calculada por Kaplan– Meier sobre los individuos de entrenamiento que cayeron en esa hoja. Esta se expresa de la siguiente manera:

$$S(t) = P(T > t).$$

Su estimador no paramétrico clásico es el de Kaplan-Meier:

$$\widehat{S}_h(t) = \prod_{t_{h,l} \le t} \left(1 - \frac{d_{h,l}}{n_{h,l}}\right),$$

donde $t_1 < t_2 < \cdots$ son los instantes ordenados en los que ocurre al menos un evento, d_l el número de eventos en t_l, y n_l el número de individuos en riesgo justo antes de t_l. Asociadas a $\widehat{S}_h$ se obtienen también:

- la función de riesgo acumulado $\widehat{H}_h(t) = -\log \widehat{S}_h(t)$ (estimador de Nelson Aalen), descrito en [11].
- el tiempo mediano de supervivencia $\widehat{t}^h_{0.5}$, definido como $\min\{t : \widehat{S}_h(t) \le 0.5\}$, cuando existe.

Para un paciente nuevo x, el árbol lo dirige hasta una hoja $h(x)$ y devuelve $\widehat{S}_{h(x)}(t)$ como predicción puntual de curva entera, no de un único valor.

5.1.3 Random Survival Forest

La extensión a bosques aleatorios, propuesta por Ishwaran en [12], conserva los mecanismos del Capítulo 4 y modifica únicamente:

- El criterio de partición, que pasa a ser el log-rank descrito arriba.
- La agregación final, que ya no es una media ni una votación mayoritaria, sino un promedio de funciones de supervivencia sobre los B árboles:

$$\widehat{S}^{B}(t \mid x) = \frac{1}{B} \sum_{b=1}^{B} \widehat{S}^{(b)}_{h_b(x)}(t),$$

  donde $h_b(x)$ es la hoja del b-ésimo árbol que contiene a x.

5.1.4 Métricas de evaluación

El acierto y la matriz de confusión no son aplicables. Las dos métricas estándar son:

1. **Índice de concordancia (1-C-index)**: el C-index de Harrell, explicado en [13], generaliza el AUC a datos censurados: estima $\mathbb{P}(\widehat{r}(x_i) > \widehat{r}(x_j) \mid T_i < T_j, \delta_i = 1)$, donde $\widehat{r}$ es el riesgo predicho y $\delta_i = 1$ indica que el evento fue observado. Toma valores en $[0, 1]$, con 0.5 correspondiente a predicción aleatoria.
2. **Continuous Ranked Probability Score (CRPS)**: equivalente al *Brier score* integrado introducido por Graf en [14], que mide la distancia integrada entre la curva de supervivencia predicha y la curva observada para cada paciente. Una versión estandarizada compara el modelo frente al modelo nulo (Kaplan–Meier marginal); valores entre 0.10 y 0.25 son habituales en este contexto, aunque dependen de la proporción de eventos observados.

5.2 Implementación en R

El análisis se ha llevado a cabo mediante el lenguaje de programación R. Todo el código desarrollado sigue el flujo habitual en proyectos de aprendizaje automático: procesamiento de los datos, ajuste del modelo y evaluación de resultados.

Primero se importarán las librerías que se usarán para el desarrollo del proyecto.

En este estudio se analiza la supervivencia hasta la recaída o muerte de pacientes ya tratados de cáncer de mama. Para ello, se usará una base de datos obtenida de la página de Kaggle.

Una vez descargados los datos, se tiene que leer el CSV para poder trabajar con los datos.

5.2.1 Procesamiento de los datos

Primero, se verá si hay valores NA en la base de datos, es decir, valores no disponibles o faltantes.

```
age    meno    size   grade   nodes     pgr      er  hormon
 0       0       0       0       0       0       0       0
 rfstime  status
  0       0
```

Como no hay valores faltantes, no se tendrá que eliminar ni rellenar ningún valor, por lo tanto se comprobará si hay datos repetidos.

```
FALSE
```

Al no haber datos repetidos, se puede pasar a estudiar el número de datos censurados: 0 y el número de eventos (recaída/muerte): 1.

```
"Número de datos censurados (0) y eventos (1)"

     0   1
   387 299
"Proporción de datos censurados (0) y eventos (1)"

       0        1
56.41399 43.58601
```

Por último, se dividirán los datos en $90\%/10\%$, donde el 90% son datos para entrenar el modelo y el 10% restante para la validación.

5.2.2 Ajuste del modelo: Árboles de Supervivencia

Primero, construiremos el árbol con todas las variables predictoras y para limitar el árbol se impondrá que cada nodo hoja debe contener al menos 15 individuos. Además, se activarán algunos parámetros para poder luego conseguir más información.

```
                         Sample size: 617
                    Number of deaths: 271
                     Number of trees: 1
          Forest terminal node size: 15
       Average no. of terminal nodes: 31
No. of variables tried at each split: 8
              Total no. of variables: 8
       Resampling used to grow trees: swor
    Resample size used to grow trees: 390
                            Analysis: RSF
                              Family: surv
                     Splitting rule: logrank *random*
       Number of random split points: 10
                          (OOB) CRPS: 429.71319374
             (OOB) standardized CRPS: 0.18116071
   (OOB) Requested performance error: 0.33700177
```

En esta salida se puede ver información como el número de individuos con el que se ha entrenado el árbol y cuáles son muertes. También se muestra el número de nodos terminales, qué criterio se ha usado para la división y diferentes errores.

Esta forma de crear árboles de supervivencia tiene una limitación, y es que se aplica el *bootstrap* sobre el conjunto de entrenamiento, es por eso que no podemos tener en cuenta los últimos errores.

Además, el modelo proporciona más información como la función de riesgo acumulado y de supervivencia, los instantes de tiempo donde ocurre una muerte/recaída, la importancia de las variables, el nodo terminal en el que caen los individuos, entre otras.

Para dibujar el árbol se usará la función `rpart()`, por lo que se necesitará volver a construir el árbol usando esa misma librería para posteriormente graficarlo.

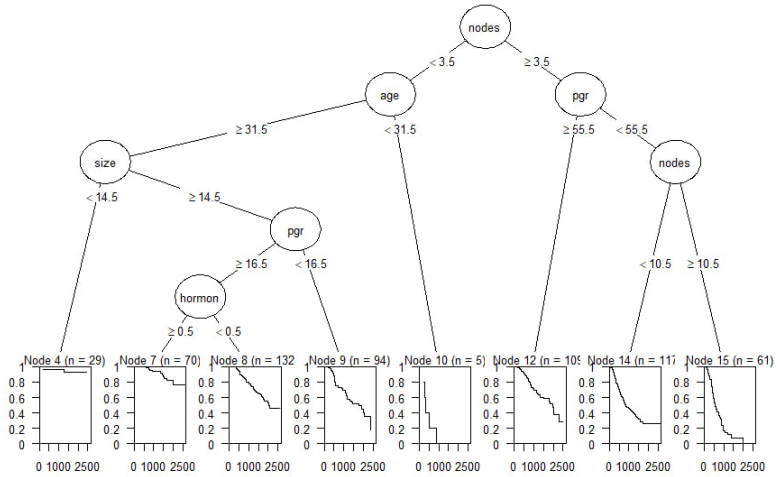


Figure 5.1: Árbol con supervivencia

En la Figura 5.1 se puede observar que el nodo raíz es *nodes* y a partir de ahí se usan más variables como *size, pgr, hormon, etc.* para seguir construyendo el árbol. Al final se puede ver que los nodos terminales, nodos hoja, son curvas de supervivencia, ya que se está buscando obtener predicciones de la curva de supervivencia.

Lo interesante es realizar predicciones usando el modelo anterior y el conjunto de prueba, de donde se obtienen los siguientes resultados.

```
Sample size of test (predict) data: 69
             Number of grow trees: 1
Average no. of grow terminal nodes: 31
       Total no. of grow variables: 8
     Resampling used to grow trees: swor
                          Analysis: RSF
                            Family: surv
                              CRPS: 578.92710663
                  standardized CRPS: 0.24406708
       Requested performance error: 0.4403344
```

Se tiene que el *Continuous Ranked Probability Score* (CRPS) es de 578.92710. Este mide el error promedio de las predicciones de supervivencia usando las observaciones *Out-Of-Bag*. También se obtiene la versión estandarizada, que es de 0.24406. Por último, tenemos el índice de concordancia, que es de 0.44033. En ambos casos, cuanto más pequeño sea, mejor.

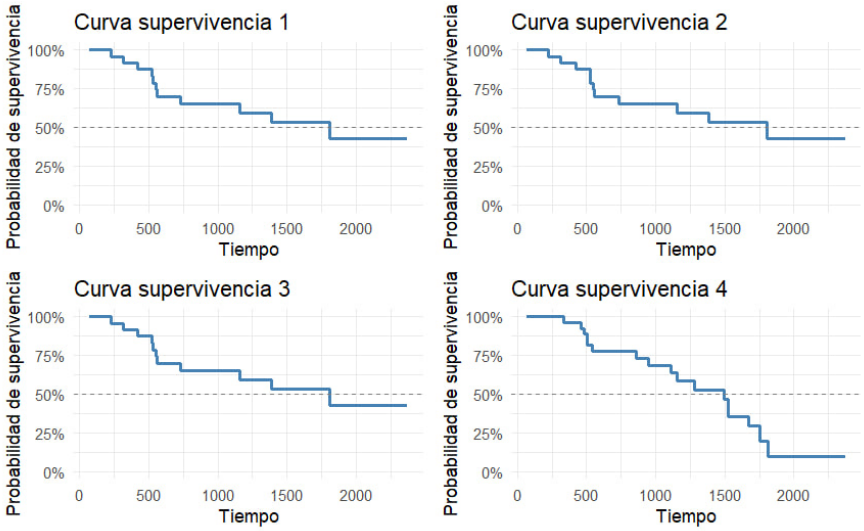


Figure 5.2: Curvas de supervivencia para 4 individuos

Del modelo de predicción, podemos sacar la función de supervivencia de cada individuo que estamos prediciendo, como se observa en la Figura 5.2, que presentan las funciones de supervivencia de los cuatro primeros, pero, si resulta necesario, se puede graficar la curva del paciente deseado.

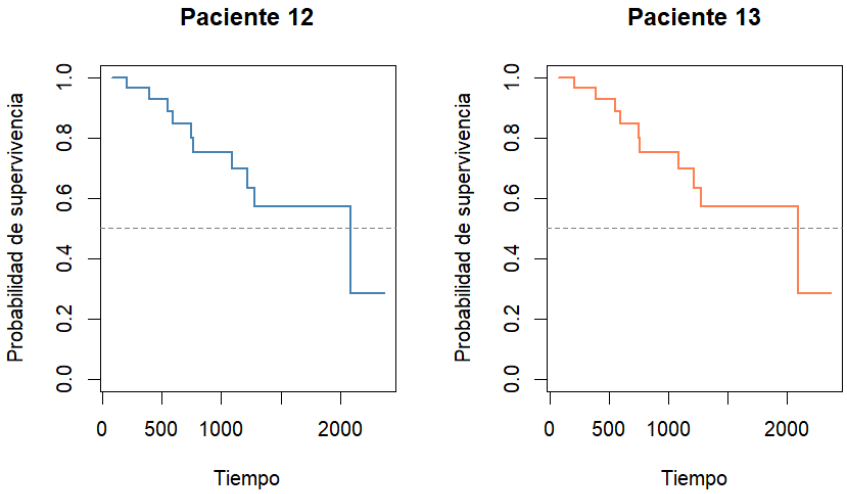


Figure 5.3: Curva supervivencia para el paciente 12 y 13

En la misma Figura 5.2 se puede apreciar que las curvas para los tres primeros son

iguales, y esto se debe a que son predichos en el mismo nodo, es decir, tienen los mismos rangos de valores para las variables predictoras del árbol.

Esto se puede comprobar tomando dos pacientes que son predichos en el mismo nodo terminal y pintando sus curvas de supervivencia. Para ver que son predichos en el mismo nodo, se usará la información del modelo explicada anteriormente y se pintarán las curvas como hicimos anteriormente. Estos resultados quedan reflajados en la Figura 5.3.

Luego, llegamos a la conclusión de que pacientes que el modelo toma como iguales según las divisiones y las variables empleadas, tienen la misma función de supervivencia.

Si no se quiere información específica de cada individuo, se puede graficar la curva de supervivencia media para las predicciones del modelo, además de la curva de supervivencia media de los datos de entrenamiento, como se muestran en las Figuras 5.4 y 5.5.

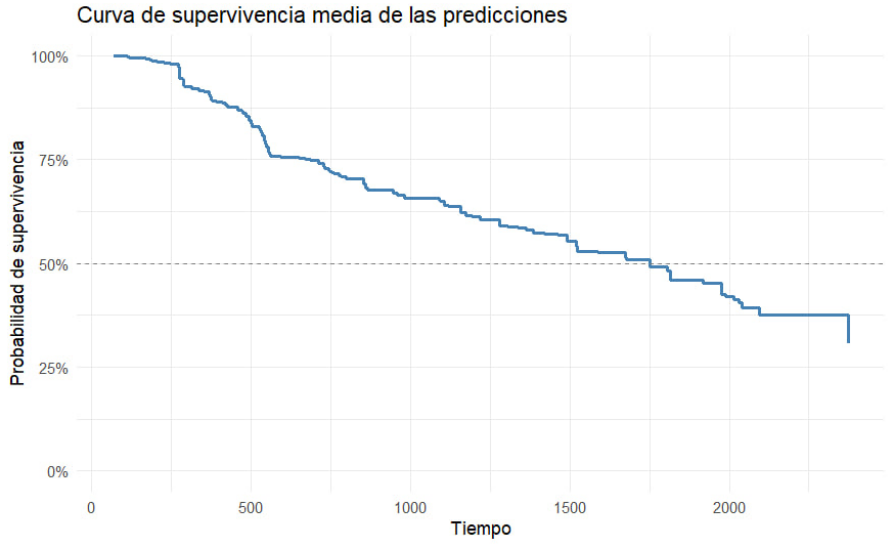


Figure 5.4: Curva supervivencia media

Como se mencionó anteriormente, el modelo aporta información sobre qué variables son más importantes, de las cuales, si se pinta un diagrama de barras, se obtendría el resultado mostrado en la Figura 5.6 .

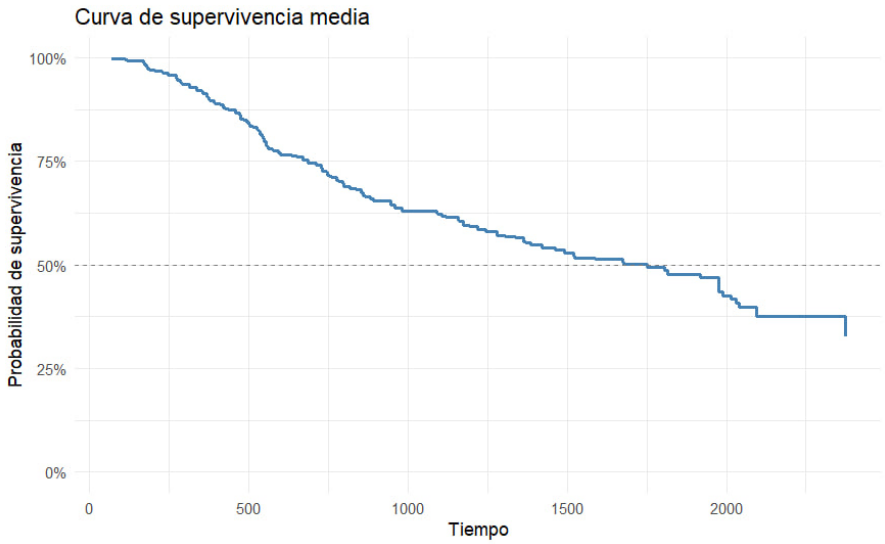


Figure 5.5: Curva supervivencia media para datos entrenamiento

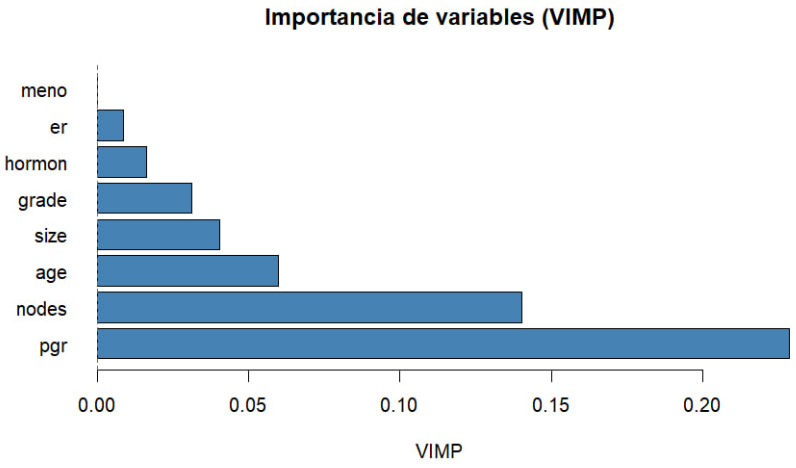


Figure 5.6: Variables importantes

Finalmente, se puede concluir que las variables *pgr*, *nodes* y *age* son las variables que mayor información aportan al modelo.

A continuación, se implementarán los bosques aleatorios de supervivencia para la misma base de datos.

5.2.3 Ajuste del modelo Random Survival Forest

De la misma manera que se hizo en la sección anterior, primero se plantea el modelo con 500 árboles y una selección aleatoria de 3 variables para realizar la división.

```
                         Sample size: 686
                    Number of deaths: 299
                     Number of trees: 500
          Forest terminal node size: 15
       Average no. of terminal nodes: 33.146
No. of variables tried at each split: 3
              Total no. of variables: 8
       Resampling used to grow trees: swor
    Resample size used to grow trees: 434
                            Analysis: RSF
                              Family: surv
                    Splitting rule: logrank *random*
       Number of random split points: 10
                         (OOB) CRPS: 393.8643411
            (OOB) standardized CRPS: 0.16036822
   (OOB) Requested performance error: 0.42729082


         Importance   Relative Imp
nodes        0.1106         1.0000
age          0.0666         0.6023
size         0.0120         0.1086
er           0.0026         0.0232
grade       -0.0028        -0.0254
meno        -0.0041        -0.0373
hormon      -0.0130        -0.1175
pgr         -0.0371        -0.3357
```

El resultado del modelo nos aporta mucha información como la importancia de las variables, el número promedio de nodos terminales, así como diferentes errores, los mismos que los modelos anteriores, que se interpretan de la misma manera. Asimismo, se puede observar que el *Random Survival Forest* llega a proporcionar una mejor predicción de los pacientes.

A continuación, en la Figura 5.7, se pintará el diagrama de barras de las variables importantes y un gráfico del error OOB dependiendo del número de árboles usados para entrenar el modelo.

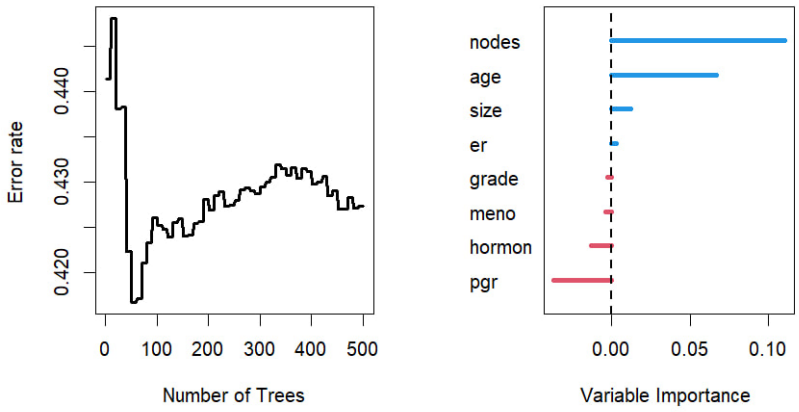


Figure 5.7: Random survival forest

Se puede observar que las variables importantes no cambian apenas respecto a considerar el modelo del árbol individual. Por otro lado, se puede analizar el número de árboles a escoger, ya que si se escogen pocos árboles, el error OOB es más pequeño, pero tampoco cambia tanto respecto a escoger 500 árboles, ya que por este lado, al haber más árboles disminuye más la varianza.

En la sección anterior vimos que para dos pacientes clasificados por el modelo como iguales, se les asociaba la misma función de supervivencia, pero para bosques de supervivencia no es así. En la Figura 5.8 se presentan las curvas de supervivencia para los mismos pacientes de la Figura 5.3 según el nuevo modelo.

Es fácil observar que las curvas no coinciden; esto es lógico ya que en un bosque aleatorio se realiza un promedio/ponderación de los datos, por lo que para bosques aleatorios de supervivencia se hará un promedio de curvas de supervivencia. A esto hay que añadir que en algunos árboles sus curvas pueden coincidir o ser distintas, por lo que salen diferentes.

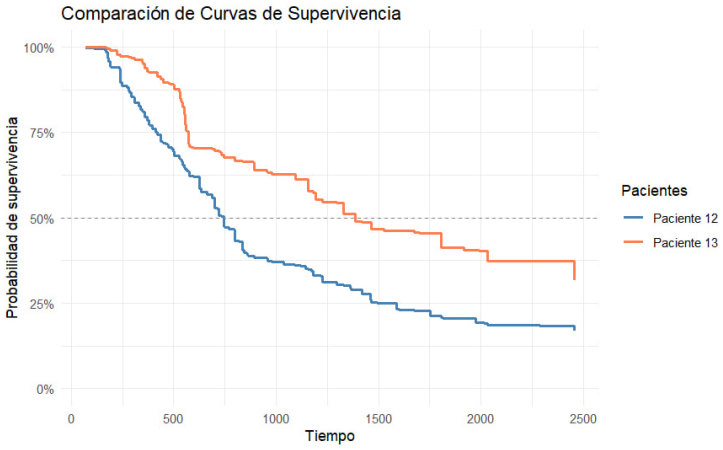


Figure 5.8: Curvas dos pacientes (Random Survival Forest)

Es interesante ver cómo el modelo discrimina para casos extremos. En este caso se tomará la variable *nodes*, que es la que tiene mayor importancia, y se ordenará de menor a mayor para posteriormente tomar el primer y último individuo y se observan las curvas de supervivencia representadas en la Figura 5.9.

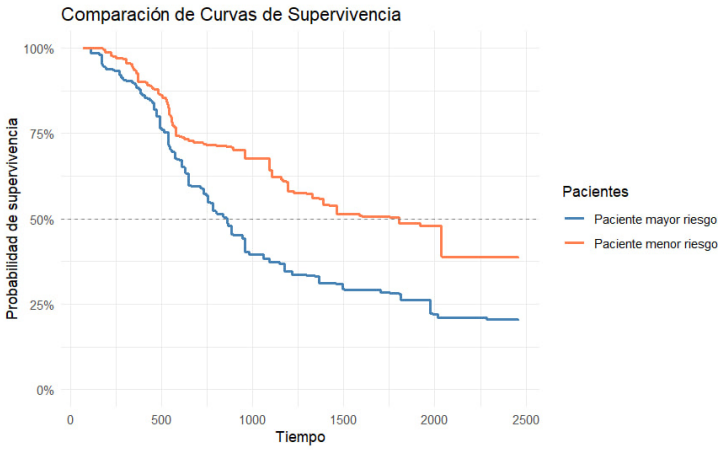


Figure 5.9: Curvas dos pacientes con mayor vs menor riesgo

Es lógico que el primer paciente, el que tiene menos ganglios, sobreviva más que el que tiene más ganglios. Esto se puede apreciar en las curvas de supervivencia.

Otra idea de modelo que se puede construir es el modelo completo del árbol, pero usando solo las variables más importantes.

5.2.4 Creación modelo completo

A este nuevo modelo se le denominará modelo completo, ya que entrenamos con todo el conjunto de datos, no se divide en entrenamiento y prueba, además de que se usará un conjunto limitado de variables.

```
                         Sample size: 686
                    Number of deaths: 299
                     Number of trees: 1
          Forest terminal node size: 15
      Average no. of terminal nodes: 28
No. of variables tried at each split: 3
              Total no. of variables: 3
      Resampling used to grow trees: swor
   Resample size used to grow trees: 434
                           Analysis: RSF
                             Family: surv
                     Splitting rule: logrank *random*
      Number of random split points: 10
                         (OOB) CRPS: 466.42455168
            (OOB) standardized CRPS: 0.18991228
  (OOB) Requested performance error: 0.48117711
```

Finalmente, la idea de la implementación es realizar un modelo que nos sirva para predecir un nuevo paciente que no esté en la base de datos, para saber cómo predice el modelo. Usando el modelo de bosque aleatorio de supervivencia, se tiene que la curva para un paciente con las siguientes características es la gráfica que se muestra en la Figura 5.10.

Table 5.1: Datos del Nuevo Paciente

Variable	Descripción	Valor
age	Edad	75
meno	Estado menopáusico	1
size	Tamaño del tumor (mm)	54
grade	Grado tumoral	1
nodes	Nodos linfáticos afectados	18
pgr	Receptores de progesterona	300
er	Receptores de estrógeno	250
hormon	Terapia hormonal	1

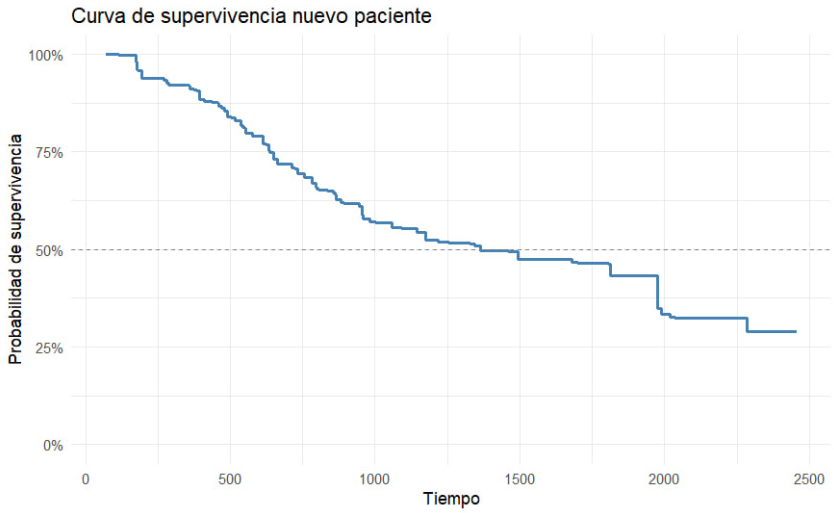


Figure 5.10: Curva supervivencia nuevo paciente

Para este nuevo paciente, tenemos que a los 1500 días la probabilidad de recaída llega al 50%.

CAPITULO

6

Conclusiones

A lo largo de este trabajo se ha desarrollado el marco teórico de los árboles de decisión individuales, tanto en su variante CHAID como en CART, y su extensión natural a los bosques aleatorios. Posteriormente, ante la naturaleza temporal y censurada de la variable respuesta en el conjunto de datos GBSG, se ha adaptado este marco al contexto del análisis de supervivencia, construyendo árboles de supervivencia y *Random Survival Forests* sobre datos clínicos reales de cáncer de mama.

Este capítulo recoge las conclusiones extraídas de dicho recorrido. Sobre la elección del marco metodológico. La primera conclusión es de naturaleza metodológica. Aunque el algoritmo CHAID resulta atractivo por su interpretabilidad estadística, al fundamentarse en contrastes de hipótesis explícitos, su aplicación directa al dataset GBSG es limitada por dos motivos. En primer lugar, la mayoría de las variables predictoras (*age*, *size*, *nodes*, *pgr*, *er*) son numéricas continuas, lo que obligaría a una discretización previa con la consiguiente pérdida de información estadística. En segundo lugar, ni CHAID ni CART en su formulación clásica gestionan adecuadamente la censura por la derecha presente en *rfstime*: tratar el tiempo de seguimiento como una variable continua ordinaria introduciría un sesgo sistemático en pacientes con censura, cuyos tiempos reales hasta la recaída son por definición desconocidos. Por ello, se ha optado por un marco específico de supervivencia, basado en el criterio de partición *log-rank*, que sí incorpora la información de censura en cada nodo. Ignorar la censura habría invalidado las conclusiones clínicas del modelo. El árbol de supervivencia construido con *rfsrc* (limitado a un único árbol con *ntree* = 1 para

emular un árbol clásico) permitió identificar las variables que el algoritmo selecciona como más discriminativas. Los nodos terminales del árbol son grupos de pacientes con perfiles clínicos homogéneos, y cada uno de ellos lleva asociada una curva de supervivencia Kaplan–Meier estimada sobre los individuos que cayeron en esa hoja. Este comportamiento se verificó explícitamente comparando las curvas de supervivencia de dos pacientes que el árbol asignaba al mismo nodo terminal: las curvas resultaron idénticas, lo cual es coherente con la lógica del modelo (un árbol predice una única función de supervivencia por hoja).

No obstante, el árbol individual presenta los problemas habituales de alta varianza: pequeñas perturbaciones en los datos de entrenamiento generan estructuras de árbol notablemente distintas, y la asignación de un paciente a un nodo es discontinua en las variables de corte. Estas limitaciones son precisamente las que motivan el paso al modelo del *Random Survival Forest*. El *Random Survival Forest*, construido con 500 árboles y criterio log-rank, confirmó las ventajas teóricas del Capítulo 4 en el contexto de supervivencia:

1. Reducción de varianza manteniendo el sesgo. El promedio de funciones de supervivencia sobre los árboles produce predicciones más estables que el árbol individual, manteniendo la flexibilidad para capturar interacciones no lineales entre covariables.

2. CRPS estandarizado de 0.16036. Esta métrica indica que el modelo reduce el error de predicción en un 16% con respecto al modelo nulo de Kaplan–Meier marginal (que asigna la misma curva a todos los pacientes). El valor obtenido se sitúa dentro del rango habitual considerado razonable en la literatura de supervivencia descrito en [14], lo cual confirma que las covariables aportan información predictiva no trivial.

3. Discriminación entre perfiles extremos. La comparación de curvas entre el paciente con menor número de ganglios afectados (*nodes*=1) y el paciente con mayor número (*nodes*=51) muestra una diferencia clínicamente coherente: los pacientes con más ganglios positivos presentan curvas significativamente más bajas, en línea con el conocimiento médico previo sobre la enfermedad.

Sobre la importancia de las variables. El análisis VIMP (*Variable Importance*), siguiendo la formulación de Ishwaran en [15], identificó en orden decreciente a *nodes*, *age* y *size* como las tres variables con mayor poder predictivo. Este resultado es relevante por dos razones:

- Confirma la coherencia clínica del modelo: el número de ganglios linfáticos axilares positivos (*nodes*) es históricamente uno de los factores pronósticos más establecidos en cáncer de mama, el tamaño del tumor (*size*) nos puede dar información de si el tumor ha podido invadir tejidos cercanos, y la edad del paciente (*age*) nos puede llegar a indicar la causa: siendo más joven suele ser más agresivo.

Sobre la predicción individual. La capacidad del RSF para devolver una curva de supervivencia personalizada para un paciente nuevo, y no un único valor puntual, constituye una de sus ventajas más importantes en el contexto clínico. Sobre el paciente sintético construido en el Capítulo 5 (mujer de 75 años, premenopáusica, tumor de 54 mm, grado 1, 18 ganglios afectados, receptores de progesterona y estrógeno elevados, en terapia hormonal), el modelo predice las siguientes probabilidades de supervivencia libre de recaída:

El tiempo mediano estimado de supervivencia libre de recaída para este paciente es de aproximadamente 1 363 días. Es interesante destacar que, pese al alto número de ganglios afectados, la curva no presenta una caída tan abrupta como podría esperarse a priori: el modelo está integrando que el paciente cuenta con receptores hormonales elevados y recibe terapia hormonal, factores con efecto protector documentado. Esta granularidad temporal permite a un clínico no solo estimar si un paciente sobrevivirá, sino cuándo es más probable que se produzca la recaída, lo cual tiene implicaciones directas sobre la frecuencia del seguimiento y la elección del tratamiento.

A pesar de las ventajas, este estudio también presenta limitaciones:

- **Tamaño muestral**: aunque 686 pacientes es un tamaño razonable, los *Random Survival Forests* se benefician de muestras mayores. En particular, las estimaciones de las curvas de supervivencia en los nodos terminales con pocos individuos están sujetas a una incertidumbre considerable.

- **Censura informativa**: el marco del RSF asume que la censura es no informativa, es decir, que los pacientes censurados tienen el mismo riesgo subyacente que los no censurados. En estudios reales esta hipótesis puede no cumplirse estrictamente.

- **Naturaleza retrospectiva**: el dataset GBSG recoge datos históricos, y los protocolos terapéuticos del cáncer de mama han evolucionado sustancialmente desde su recolección.

CAPITULO

7

Anexos

7.1 Derivaciones Matemáticas y Demostraciones

7.1.1 Varianza del Bosque Aleatorio

Proposición 7.1.1. Sean $T_1, T_2, \ldots, T_B$ variables aleatorias e idénticamente distribuidas (i.d.) con varianza σ^2 y correlación par a par ρ. La varianza del promedio del conjunto (bosque) viene dada por:

$$\text{Var}\left[\frac{1}{B}\sum_{b=1}^{B} T_b(x)\right] = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 \tag{7.1}$$

Proof. Se considera el bosque como el promedio de las predicciones de B árboles. Utilizando las propiedades de la varianza para una suma de variables aleatorias dependientes:

$$\begin{aligned}\text{Var}\left(\frac{1}{B}\sum_{b=1}^{B} T_b\right) &= \frac{1}{B^2}\text{Var}\left(\sum_{b=1}^{B} T_b\right) \\ &= \frac{1}{B^2}\left[\sum_{b=1}^{B}\text{Var}(T_b) + \sum_{b=1}^{B}\sum_{k\neq b}^{B}\text{Cov}(T_b, T_k)\right]\end{aligned}$$

Dado que los árboles son i.d., $\text{Var}(T_b) = \sigma^2$. La covarianza se expresa mediante la correlación como $\text{Cov}(T_b, T_k) = \rho\sigma^2$. Al sumar los B términos de varianza y los $B(B-1)$

términos de covarianza:

$$\begin{aligned}\text{Var}\left(\frac{1}{B}\sum_{b=1}^{B}T_b\right) &= \frac{1}{B^2}\left[B\sigma^2 + B(B-1)\rho\sigma^2\right] \\ &= \frac{\sigma^2}{B} + \frac{B-1}{B}\rho\sigma^2 \\ &= \frac{\sigma^2}{B} + \left(1-\frac{1}{B}\right)\rho\sigma^2 \\ &= \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2\end{aligned}$$

Queda así demostrada la reducción de la varianza en función de la descorrelación de los árboles. □

7.1.2 Sesgo del Bosque Aleatorio

Proposición 7.1.2. Bajo la asunción de que los árboles T_b son idénticamente distribuidos, el sesgo del estimador del bosque es idéntico al sesgo de cualquier árbol individual componente.

Proof. Sea $\hat{f}^B(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$ el estimador del bosque. Aplicando el operador esperanza y la propiedad de linealidad:

$$E[\hat{f}^B(x)] = E\left[\frac{1}{B}\sum_{b=1}^{B}T_b(x)\right] = \frac{1}{B}\sum_{b=1}^{B}E[T_b(x)] \tag{7.2}$$

Como los árboles son i.d., todos comparten la misma esperanza $E[T_b(x)] = \mu$. Entonces:

$$E[\hat{f}^B(x)] = \frac{1}{B}(B\cdot\mu) = \mu \tag{7.3}$$

Definimos el sesgo como la diferencia entre la esperanza de la predicción y el valor real $f(x)$. Bajo la asunción de que la esperanza del árbol individual es μ, el sesgo del bosque es:

$$\text{Sesgo}(\hat{f}^B(x)) = \mu - f(x) \tag{7.4}$$

Este resultado es idéntico al sesgo de un árbol individual, lo que confirma que el promediado en *Bagging* no reduce el sesgo del modelo, descrito en [16]. □

7.1.3 Ajuste de Bonferroni para Variables Nominales (Stirling)

En el algoritmo CHAID, el ajuste de Bonferroni es fundamental para corregir la inflación del error Tipo I debido a las múltiples comparaciones. Cuando se analiza una variable

predictora nominal con k categorías que se agrupan en c categorías finales, el factor de ajuste B debe representar el número total de formas posibles de realizar dicha partición.

Matemáticamente, el número de formas de particionar un conjunto de k elementos en c subconjuntos no vacíos (donde el orden de los subconjuntos no importa) se define mediante los números de Stirling de segunda especie, denotados como $S(k, c)$ o $\left\{ {k \atop c} \right\}$.

La fórmula general para estos números, derivada mediante el principio de inclusión-exclusión, es:

$$S(k,c) = \frac{1}{c!} \sum_{j=0}^{c} (-1)^j \binom{c}{j} (c-j)^k \tag{7.5}$$

Para demostrar la equivalencia con la fórmula utilizada en el cuerpo del trabajo, desarrollamos el coeficiente binomial $\binom{c}{j}$ dentro de la sumatoria:

$$B = \frac{1}{c!} \sum_{j=0}^{c-1} (-1)^j \left[\frac{c!}{j!(c-j)!} \right] (c-j)^k \tag{7.6}$$

Al simplificar el término $c!$ que aparece tanto en el numerador como en el denominador fuera de la sumatoria, y ajustando el límite superior de la sumatoria a $c-1$ (puesto que para $j = c$ el término $(c-j)^k$ es igual a 0), obtenemos la expresión compacta empleada en [1] e implementada en la documentación técnica descrita en [2]:

$$B = \sum_{j=0}^{c-1} (-1)^j \frac{(c-j)^k}{j!(c-j)!} \tag{7.7}$$

Por lo que el uso de esta fórmula en el algoritmo CHAID garantiza que el p-valor sea penalizado proporcionalmente al tamaño del espacio de búsqueda de agrupaciones posibles. Sin este ajuste basado en los números de Stirling, el algoritmo tendería a seleccionar variables con muchas categorías simplemente por azar estadístico, invalidando la capacidad predictiva del árbol de decisión.

7.2 Códigos y Scripts en R

En esta sección se detallan los scripts utilizados para la generación de los gráficos y modelos presentados a lo largo de la memoria.

A continuación, se detallan los procedimientos programados en R para la visualización de las fronteras de decisión y superficies de predicción.

7. ANEXOS

El script diseñado para el caso de clasificación (modelo CART con el dataset iris) se presenta a continuación:

```
data(iris)
library(rpart)
library(ggplot2)

# 1. Crear el modelo de clasificacion
# Usamos solo dos variables (Petal.Length y Petal.Width) para poder graficar en 2D
modelo_clase <- rpart(Species ~ Petal.Length + Petal.Width, data = iris, method = "class")

# 2. Crear una rejilla (grid) para el plano
res <- 0.02
grid <- expand.grid(
  Petal.Length = seq(min(iris$Petal.Length), max(iris$Petal.Length), by = res),
  Petal.Width = seq(min(iris$Petal.Width), max(iris$Petal.Width), by = res)
)

# 3. Predecir la clase para cada punto del plano
grid$Species <- predict(modelo_clase, grid, type = "class")

# 4. Graficar
ggplot() +
  geom_tile(data = grid, aes(x = Petal.Length, y = Petal.Width, fill = Species), alpha = 0.3) +
  geom_point(data = iris, aes(x = Petal.Length, y = Petal.Width, color = Species)) +
  labs(title = "CART: Fronteras de Decision (Clasificacion)",
       subtitle = "Las lineas rectas muestran cómo el arbol segmenta el plano") +
  theme_minimal()
```

De igual modo, se especifica el código para el análisis de regresión.

```
# 1. Crear el modelo de regresion
modelo_reg <- rpart(mpg ~ hp + wt, data = mtcars, method = "anova")

# 2. Crear la rejilla
grid_reg <- expand.grid(
  hp = seq(min(mtcars$hp), max(mtcars$hp), length.out = 100),
```

```
  wt = seq(min(mtcars$wt), max(mtcars$wt), length.out = 100)
)

# 3. Predecir el valor continuo (mpg)
grid_reg$mpg <- predict(modelo_reg, grid_reg)

# 4. Graficar
ggplot() +
  geom_tile(data = grid_reg, aes(x = hp, y = wt, fill = mpg)) +
  geom_point(data = mtcars, aes(x = hp, y = wt), color =
  ↪  "white", size = 2) +
  scale_fill_viridis_c() +
  labs(title = "CART: Superficie de Predicción (Regresión)",
       subtitle = "Cada color representa un valor constante
       ↪  predicho por una hoja del árbol",
       fill = "MPG Predicho") +
  theme_dark()
```

Para realizar el árbol de regresión de la Figura 2.2 usado en el Capítulo 2 se utilizó la base de datos de Kaggle que aporta el precio de una vivienda en función de muchos parámetros.

```
data<-read.csv("data/home-prices.csv")
datos<-data[,-c(8,9,10,11,12)]
head(datos)
dim(datos)
set.seed(123) # Semilla para la reproducibilidad

# Creamos una muestra de los indices de un 66% de las filas de
↪  los datos
indice <- sample(1:nrow(datos), size = 0.66 * nrow(datos))

# Creamos los dos conjuntos de datos de entrenamiento y prueba
train_data <- datos[indice, ]
test_data  <- datos[-indice, ]

modelo_regresion<- rpart(
  formula = price ~ .,
  data = train_data,
  method = "anova",
  control = rpart.control(
    maxdepth = 3
  )
)
```

```r
modelo_regresion$method <- "regresion"

# 2. Convertir a party
modelo_reg_party <- as.party(modelo_regresion)

# 3. Graficar para regresion
plot(modelo_reg_party,
     terminal_panel = node_boxplot(modelo_reg_party,
                                   id = FALSE,
                                   mainlab = "",
                                   col = "blue"),
     tp_args = list(id = FALSE),
     inner_panel = node_inner(modelo_reg_party,
                              id = FALSE,
                              pval = FALSE),
     gp = gpar(fontsize = 9))
```

Para realizar el árbol de clasificación de la Figura 2.1 usado en el Capítulo 2 se utilizó la base de datos *PimaIndiansDiabetes* incluida en el software de R.

```r
# creacion árbol de decision

library(mlbench)
data(PimaIndiansDiabetes)
status<-ifelse(PimaIndiansDiabetes$diabetes=="pos",1,0)
 ↪  #cambiamos la variable objetivo
PimaIndiansDiabetes$diabetes<-as.factor(status)

set.seed(123) # Semilla para la reproducibilidad

# Creamos una muestra de los índices de un 66% de las filas de
 ↪  nuestros datos
indice <- sample(1:nrow(PimaIndiansDiabetes), size = 0.66 *
 ↪  nrow(PimaIndiansDiabetes))

# Creamos los dos conjuntos de datos de entrenamiento y prueba
train_data <- PimaIndiansDiabetes[indice, ]
test_data  <- PimaIndiansDiabetes[-indice, ]

modelo_gini_3<- rpart(
  formula = diabetes ~ .,
  data = train_data,
  method = "class",
  parms = list(split = "gini"),
```

```
  control = rpart.control(
    maxdepth = 3
  )
)
modelo_gini_3$method <- "class"

# 2. Convertir a party
modelo_party <- as.party(modelo_gini_3)
plot(modelo_party,
     terminal_panel = node_barplot(modelo_party,
                                   beside = TRUE,
                                   col = c("red", "green"),
                                   id = FALSE),
     tp_args = list(
       id = FALSE,
       format = function(node) format_node(node, user_labels =
       ↪  FALSE, id = FALSE)
     ),
     inner_panel = node_inner(modelo_party,
                              id = FALSE,
                              pval = FALSE),
     gp = gpar(fontsize = 9)
)
```

Para generar el gráfico de la diana, representado en la Figura 2.3 también usado en el Capítulo 2 se desarrolló el siguiente código.

```
library(ggplot2)
library(gridExtra)

# Funcion para generar los puntos en la diana
generar_diana <- function(bias_val, var_val, titulo) {
  set.seed(123)
  n <- 30
  # El centro de la diana es (0,0)
  # El bias desplaza el centro de los puntos
  # La varianza dispersa los puntos
  puntos <- data.frame(
    x = rnorm(n, mean = bias_val, sd = var_val),
    y = rnorm(n, mean = bias_val, sd = var_val)
  )

  ggplot(puntos, aes(x, y)) +
    # Dibujamos los circulos de la diana
```

```
    annotate("path", x=0+1.5*cos(seq(0,2*pi,length.out=100)),
    ↪  y=0+1.5*sin(seq(0,2*pi,length.out=100)), color="grey") +
    annotate("path", x=0+1*cos(seq(0,2*pi,length.out=100)),
    ↪  y=0+1*sin(seq(0,2*pi,length.out=100)), color="grey") +
    annotate("path", x=0+0.5*cos(seq(0,2*pi,length.out=100)),
    ↪  y=0+0.5*sin(seq(0,2*pi,length.out=100)), color="grey") +
    geom_point(color="red", alpha=0.6, size=2) +
    coord_fixed(xlim = c(-2, 2), ylim = c(-2, 2)) +
    theme_minimal() +
    labs(title = titulo, x = "", y = "") +
    theme(axis.text = element_blank(), panel.grid =
    ↪  element_blank())
}

# Creamos los 4 escenarios
p1 <- generar_diana(0, 0.12, "Bajo Sesgo, Baja Varianza")
p2 <- generar_diana(0, 0.5, "Bajo Sesgo, Alta Varianza")
p3 <- generar_diana(1, 0.12, "Alto Sesgo, Baja Varianza")
p4 <- generar_diana(1, 0.5, "Alto Sesgo, Alta Varianza")

# Los juntamos todos en una imagen
grid.arrange(p1, p2, p3, p4, ncol=2)
```

Finalmente, se desarrollan los códigos usados en el Capítulo 5 para los distintos modelos y gráficos.

```
library(REdaS)
library(rpart)
library(rpart.plot)
library(partykit)
library(pROC)
library("ranger")
library(randomForest)
library(caret)
library(rsample)
library(randomForestSRC)
library(ggplot2)
library(ggRandomForests)
library(survival)
library(patchwork)
```

Para el procesamiento de los datos se usaron los siguientes códigos.

```r
# Ver conteo de NAs por columna
colSums(is.na(datos))
```

```r
# Devuelve TRUE si hay algún duplicado, FALSE si todo es único
any(duplicated(datos))
```

```r
# Contar el número exacto de casos
tabla_frecuencias <- table(datos$status)
print("Número de datos censurados (0) y eventos (1)")
print(tabla_frecuencias)

# Ver la proporción / porcentaje
proporciones <- prop.table(tabla_frecuencias) * 100
print("Proporción de datos censurados (0) y eventos (1)")
print(proporciones)
```

```r
set.seed(123) # Semilla para la reproducibilidad
datos$rfstime <- as.numeric(as.character(datos$rfstime))
datos$status  <- as.numeric(as.character(datos$status))

# Creamos una muestra de los índices de un 90% de datos
indice <- sample(1:nrow(datos), size = 0.90 * nrow(datos))

# Creamos los dos conjuntos de datos de entrenamiento y prueba
train_data <- datos[indice, ]
test_data  <- datos[-indice, ]
```

A continuación, se presentan los códigos para los árboles de supervivencia junto con los gráficos de las funciones de supervivencia.

```r
set.seed(123)
# Árbol único de supervivencia
modelo_arbol <- rfsrc(
  Surv(rfstime, status) ~ age+meno+size+grade+nodes+pgr+er+hormon,
  data = train_data,
  ntree = 1,
  mtry = ncol(train_data) - 1,  # usar todas las variables
  nodesize = 15,
  forest = TRUE ,
  membership = TRUE,
  importance=TRUE,
  var.used   = "all.trees"

)
modelo_arbol
```

```r
# Funcion de riesgo acumulado estimada para cada sujeto. H(t)
 ↪  evaluada en t mediano
modelo_arbol$predicted
# Funcion de supervivencia por sujeto matriz con tiempos t de
 ↪  muerte recaida
modelo_arbol$survival
# Funcion de riesgo acumulado estimada en cada tiempo (la H)
 ↪  tiempos t de muerte recaida
modelo_arbol$chf
# Los tiempos donde ocurre la recaída/muerte
modelo_arbol$time.interest
# Importancia de las variables cambiando los datos de una
 ↪  variable
modelo_arbol$importance
# Error oob del modelo
modelo_arbol$err.rate
# Frecuencia de las variables usadas
modelo_arbol$var.used
# Nodo final al que pertenece cada observacion
#modelo_arbol$membership
```

```r
modelo_rpart <- rpart(
  Surv(rfstime, status) ~ age+meno+size+grade+nodes+pgr+er+hormon,
  data = train_data,
  method = "exp",      #"exp" le indica a rpart que use Log-Rank
  control = rpart.control(minsplit = 15)
)

# Convertimos el árbol a formato 'party'
arbol_graficable <- as.party(modelo_rpart)

# Lo pintamos con las curvas de supervivencia en los nodos
plot(arbol_graficable,
     gp = gpar(fontsize = 8),
     inner_panel = node_inner(arbol_graficable, id = FALSE))
```

```r
predicciones <- predict(modelo_arbol, newdata =test_data)
predicciones
```

```r
par(mfrow = c(2, 2))

df_surv_pred_1 <- data.frame(
  tiempo = modelo_arbol$time.interest,
  surv   = predicciones$survival[1,]
)
```

```
7
8  p1<-ggplot(df_surv_pred_1, aes(x = tiempo, y = surv)) +
9    geom_step(color = "steelblue", linewidth = 1) +
10   geom_hline(yintercept = 0.5, linetype = "dashed", color =
     ↪  "gray50") +
11   scale_y_continuous(limits = c(0, 1), labels = scales::percent)
     ↪  +
12   labs(title = "Curva supervivencia 1",
13        x = "Tiempo", y = "Probabilidad de supervivencia") +
14   theme_minimal()
15 df_surv_pred_2 <- data.frame(
16   tiempo = modelo_arbol$time.interest,
17   surv   = predicciones$survival[2,]
18 )
19
20 p2<-ggplot(df_surv_pred_2, aes(x = tiempo, y = surv)) +
21   geom_step(color = "steelblue", linewidth = 1) +
22   geom_hline(yintercept = 0.5, linetype = "dashed", color =
     ↪  "gray50") +
23   scale_y_continuous(limits = c(0, 1), labels = scales::percent)
     ↪  +
24   labs(title = "Curva supervivencia 2",
25        x = "Tiempo", y = "Probabilidad de supervivencia") +
26   theme_minimal()
27 df_surv_pred_3 <- data.frame(
28   tiempo = modelo_arbol$time.interest,
29   surv   = predicciones$survival[3,]
30 )
31
32 p3<-ggplot(df_surv_pred_3, aes(x = tiempo, y = surv)) +
33   geom_step(color = "steelblue", linewidth = 1) +
34   geom_hline(yintercept = 0.5, linetype = "dashed", color =
     ↪  "gray50") +
35   scale_y_continuous(limits = c(0, 1), labels = scales::percent)
     ↪  +
36   labs(title = "Curva supervivencia 3",
37        x = "Tiempo", y = "Probabilidad de supervivencia") +
38   theme_minimal()
39 df_surv_pred_4 <- data.frame(
40   tiempo = modelo_arbol$time.interest,
41   surv   = predicciones$survival[4,]
42 )
43
44 p4<-ggplot(df_surv_pred_4, aes(x = tiempo, y = surv)) +
45   geom_step(color = "steelblue", linewidth = 1) +
```

```
  geom_hline(yintercept = 0.5, linetype = "dashed", color =
  ↪ "gray50") +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent)
  ↪ +
  labs(title = "Curva supervivencia 4",
       x = "Tiempo", y = "Probabilidad de supervivencia") +
  theme_minimal()
(p1 + p2)/(p3+p4)

par(mfrow = c(1,1))
```

```
df_surv_pred <- data.frame(
  tiempo = modelo_arbol$time.interest,
  surv   = colMeans(predicciones$survival)
)

ggplot(df_surv_pred, aes(x = tiempo, y = surv)) +
  geom_step(color = "steelblue", linewidth = 1) +
  geom_hline(yintercept = 0.5, linetype = "dashed", color =
  ↪ "gray50") +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent)
  ↪ +
  labs(title = "Curva de supervivencia media de las
  ↪ predicciones",
       x = "Tiempo", y = "Probabilidad de supervivencia") +
  theme_minimal()

```

```
# Verificamos que caen en el mismo nodo
modelo_arbol$membership[12,]==modelo_arbol$membership[13,]

# Extraemos los tiempos (son los mismos para todo el modelo)
tiempos <- modelo_arbol$time.interest

#  Creamos un único dataframe con AMBAS curvas en formato "largo"
df_surv_dos <- data.frame(
  tiempo = c(tiempos, tiempos),
  surv = c(modelo_arbol$survival[12, ],
  ↪ modelo_arbol$survival[13, ]),
  Curva = c(rep("Paciente 12", length(tiempos)), rep("Paciente
  ↪ 13", length(tiempos)))
)

# Pintamos el gráfico con ggplot
ggplot(df_surv_dos, aes(x = tiempo, y = surv, color = Curva)) +
```

```
  geom_step(linewidth = 1) +  # Quitamos el color fijo de aquí
  ↪ para que use el de aes()
  geom_hline(yintercept = 0.5, linetype = "dashed", color =
  ↪ "gray50") +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent)
  ↪ +
  scale_color_manual(values = c("steelblue", "coral")) + # Aquí
  ↪ eliges los dos colores
  labs(
    title = "Comparación de Curvas de Supervivencia",
    x = "Tiempo",
    y = "Probabilidad de supervivencia",
    color = "Pacientes"
  ) +
  theme_minimal()
```

```
df_surv_media <- data.frame(
  tiempo = modelo_arbol$time.interest,
  surv   = colMeans(modelo_arbol$survival)
)

ggplot(df_surv_media, aes(x = tiempo, y = surv)) +
  geom_step(color = "steelblue", linewidth = 1) +
  geom_hline(yintercept = 0.5, linetype = "dashed", color =
  ↪ "gray50") +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent)
  ↪ +
  labs(title = "Curva de supervivencia media",
       x = "Tiempo", y = "Probabilidad de supervivencia") +
  theme_minimal()
```

```
# Ordenar e imprimir
vimp <- sort(modelo_arbol$importance, decreasing = TRUE)

barplot(vimp,
        horiz = TRUE,       # barras horizontales
        las   = 1,
        col   = ifelse(vimp > 0, "steelblue", "tomato"),
        main  = "Importancia de variables (VIMP)",
        xlab  = "VIMP")
abline(v = 0, lty = 2, col = "gray40")
```

Los códigos usados para la construcción de los random survival forest se plasman a continuación.

```r
set.seed(256)
modelo_rsf<-rfsrc(Surv(rfstime,status)~age+meno+size+grade+nodes
 ↪  +pgr+er+hormon,
                   data=datos,
                   ntree=500,
                   mtry = 3,
                   bootstrap = "by.root",
                   splitrule="logrank",
                   importance=TRUE,
                 membership = TRUE,
                 var.used   = "all.trees")

modelo_rsf
plot(modelo_rsf)
```

```r
#  Extraemos los tiempos (son los mismos para todo el modelo)
tiempos <- modelo_rsf$time.interest

#  Creamos un único dataframe con AMBAS curvas en formato "largo"
df_surv_rsf<- data.frame(
  tiempo = c(tiempos, tiempos),
  surv = c(modelo_rsf$survival[12, ], modelo_rsf$survival[13, ]),
  Curva = c(rep("Paciente 12", length(tiempos)), rep("Paciente
   ↪  13", length(tiempos)))
)

#  Pintamos el gráfico con ggplot
ggplot(df_surv_rsf, aes(x = tiempo, y = surv, color = Curva)) +
  geom_step(linewidth = 1) +  # Quitamos el color fijo de aquí
   ↪  para que use el de aes()
  geom_hline(yintercept = 0.5, linetype = "dashed", color =
   ↪  "gray50") +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent)
   ↪  +
  scale_color_manual(values = c("steelblue", "coral")) + # Aquí
   ↪  eliges los dos colores
  labs(
    title = "Comparación de Curvas de Supervivencia",
    x = "Tiempo",
    y = "Probabilidad de supervivencia",
    color = "Pacientes"
  ) +
  theme_minimal()
```

```r
# Buscamos el paciente con más nodos y el que menos nodos tenga
```

```r

# Ordenar de menor a mayor (Ascendente)
datos_ordenados <- data[order(data$nodes), ]
pos1<-head(datos_ordenados,1)["X"][1,]
pos2<-tail(datos_ordenados,1)["X"][1,]
#  Extraemos los tiempos (son los mismos para todo el modelo)
tiempos <- modelo_rsf$time.interest

#  Creamos un único dataframe con AMBAS curvas en formato "largo"
df_surv_comp <- data.frame(
  tiempo = c(tiempos, tiempos),
  surv = c(modelo_rsf$survival[pos1, ],
   ↪  modelo_rsf$survival[pos2, ]),
  Curva = c(rep("Paciente menor riesgo", length(tiempos)),
   ↪  rep("Paciente mayor riesgo", length(tiempos)))
)

#  Pintamos el gráfico con ggplot
ggplot(df_surv_comp, aes(x = tiempo, y = surv, color = Curva)) +
  geom_step(linewidth = 1) +  # Quitamos el color fijo de aquí
   ↪  para que use el de aes()
  geom_hline(yintercept = 0.5, linetype = "dashed", color =
   ↪  "gray50") +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent)
   ↪  +
  scale_color_manual(values = c("steelblue", "coral")) + # Aquí
   ↪  eliges los dos colores
  labs(
    title = "Comparación de Curvas de Supervivencia",
    x = "Tiempo",
    y = "Probabilidad de supervivencia",
    color = "Pacientes"
  ) +
  theme_minimal()
```

```r
set.seed(222)

datos_modelo_completo <- datos[, c("age", "nodes", "pgr",
 ↪  "rfstime", "status")]

# Árbol único de supervivencia
modelo_arbol_completo <- rfsrc(
  Surv(rfstime, status) ~ nodes+pgr+age,
  data = datos_modelo_completo,
  ntree = 1,
```

```
  mtry = ncol(datos_modelo_completo) - 1,
  nodesize = 15,
  forest = TRUE ,
  membership = TRUE,
  importance=TRUE,
  var.used   = "all.trees"

)
modelo_arbol_completo

modelo_rpart_completo <- rpart(
  Surv(rfstime, status) ~ nodes+pgr+age,
  data = datos_modelo_completo,
  method = "exp",
  control = rpart.control(minsplit = 15) # Tu tamaño de nodo
   ↪  (nodesize)
)

# Convertimos el árbol a formato 'party'
# Esta conversión SÍ funciona directamente porque rpart es
 ↪  compatible
arbol_graficable_completo <- as.party(modelo_rpart_completo)

# Lo pintamos con las curvas de supervivencia en los nodos
# Esto quita los números de los rectángulos y deja solo los
 ↪  nombres de las variables
plot(arbol_graficable_completo,
     gp = gpar(fontsize = 8),
     inner_panel = node_inner(arbol_graficable_completo, id =
      ↪  FALSE))
```

```
#  Hacer predicciones con datos nuevos
datos_nuevo_paciente<- data.frame(
  age=75,
  meno=1,
  size=54,
  grade=1,
  nodes=18,
  pgr=300,
  er=250,
  hormon=1
)

# Ver el resultado
```

```r
print(datos_nuevo_paciente)
predicciones <- predict(modelo_rsf, newdata
 ↪  =datos_nuevo_paciente)

df_surv <- data.frame(
  tiempo = modelo_rsf$time.interest,
  surv   = predicciones$survival[1,]
)

ggplot(df_surv, aes(x = tiempo, y = surv)) +
  geom_step(color = "steelblue", linewidth = 1) +
  geom_hline(yintercept = 0.5, linetype = "dashed", color =
   ↪  "gray50") +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent)
   ↪  +
  labs(title = "Curva de supervivencia nuevo paciente",
       x = "Tiempo", y = "Probabilidad de supervivencia") +
  theme_minimal()

```

Bibliografía

[1] Gordon V. Kass. An exploratory technique for investigating large quantities of categorical data. *Applied Statistics*, 29(2):119–127, 1980. 14, 19, 69

[2] IBM Corporation. *IBM SPSS Decision Trees 27*. IBM Corporation, Armonk, NY, 2020. 19, 69

[3] Leo Breiman, Jerome Friedman, Richard Olshen, and Charles Stone. *Classification and Regression Trees*. Wadsworth, 1984. 29

[4] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. Springer, 2 edition, 2009. 37, 38, 44

[5] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001. 41, 44

[6] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 1979. 43

[7] Patrick Royston and Douglas G. Altman. External validation of a cox prognostic model: principles and methods. *BMC Medical Research Methodology*, 13(33), 2013. 47

[8] Patrick Royston. German breast cancer study group (GBSG) dataset. `https://www.kaggle.com/datasets/...`, 2023. 2/02/2026. 47

[9] Antonio Ciampi, Johanne Thiffault, Jean-Pierre Nakache, and Bernard Asselain. Stratification by stepwise regression, correspondence analysis and recursive partition: A comparison of three methods of analysis for survival data with covariates. *Computational Statistics & Data Analysis*, 4(3):185–204, 1986. 49

[10] Michael LeBlanc and John Crowley. Survival trees by goodness of split. *Journal of the American Statistical Association*, 88(422):457–467, 1993. 49

[11] Wayne Nelson. Theory and applications of hazard plotting for censored failure data. *Technometrics*, 14(4):945–966, 1972. 50

[12] Hemant Ishwaran, Udaya B. Kogalur, Eugene H. Blackstone, and Michael S. Lauer. Random survival forests. *The Annals of Applied Statistics*, 2(3):841–860, 2008. 51

[13] Frank E. Harrell, Robert M. Califf, David B. Pryor, Kerry L. Lee, and Robert A. Rosati. Evaluating the yield of medical tests. *Journal of the American Medical Association*, 247(18):2543–2546, 1982. 51

[14] Erika Graf, Claudia Schmoor, Willi Sauerbrei, and Martin Schumacher. Assessment and comparison of prognostic classification schemes for survival data. *Statistics in Medicine*, 18(17-18):2529–2545, 1999. 51, 64

[15] Hemant Ishwaran. Variable importance in binary regression trees and forests. *Electronic Journal of Statistics*, 1:519–537, 2007. 64

[16] Gilles Louppe. *Understanding Random Forests: From Theory to Practice*. PhD thesis, University of Liège, Belgium, 2014. PhD dissertation. 68